

GTOS Handbook - Delivery Playbooks, Standards, Blueprints, and Scenarios

GTOS Enterprise Engineering Handbook - Final Edition 1.0

Source Chapters

- ADR-0001 — Markdown Is the Canonical Source
- ADR-0002 — Retain Canon Terminology
- ADR-0003 — Ontology Precedes Event Architecture
- ADR-0004 — Separate Ontology and Epistemology
- ADR-0005 — Domain Ownership over Shared Database
- ADR-0006 — Recommendations Are Not Decisions
- ADR-0007 — Intent Gates Execution
- ADR-0008 — Append-Only Correction
- ADR-0009 — Static Portable Publication
- ADR-0010 — Machine-Readable Registries
- Architecture Decision Records
- Architecture Quality Gates
- Architecture Review Checklist
- Conformance Levels
- Implementation Evidence Catalogue
- Normative Language
- Release Checklist
- Definition of Ready
- Vertical Slice Delivery Guide
- Architecture-Aware Backlog Decomposition
- Contract Testing Guide
- Production Readiness Review
- Cutover Command Center Guide
- Partner Certification Guide
- Observability Dashboard Design
- Security Verification Guide

- Data Pipeline Engineering Guide
- AI Release Engineering Guide
- Performance and Capacity Engineering
- Post-Launch Stabilization Guide
- Ninety-Day Domain Onboarding Roadmap
- Architecture Design Review Facilitation
- Command Handler Reference Flow
- Event Consumer Reference Flow
- Business Recovery Exercise
- Architecture Documentation Standard
- Architecture Debt Management
- GTOS Team Ceremonies
- Quality Assurance Strategy
- GTOS Project Plan Template
- Engineer Onboarding Guide
- Engineering Acceptance and Handover
- Engineering Execution Guides
- GTOS Delivery Handbook
- Playbook — AI Use-Case Qualification
- Playbook — Architecture Decision Review
- Playbook — Data Product Onboarding
- Playbook — Domain Discovery
- Playbook — Event Storming and State Modeling
- Playbook — Incident Learning
- Playbook — Migration and Cutover
- Playbook — Partner Onboarding
- Playbook — Release Readiness
- Playbook — Security Threat Modeling
- Playbook — SLO and Runbook Design
- Playbook — Truth Ownership Workshop
- Delivery Roles and Responsibilities
- Stage 0 — Mandate and Scope
- Stage 1 — Reference Alignment
- Stage 2 — Business and Domain Design
- Stage 3 — Contract and Data Design
- Stage 4 — Security, AI, and Operational Design
- Stage 5 — Build and Verification

- Stage 6 — Controlled Launch
- Stage 7 — Operate and Evolve
- Work Product Catalogue
- AI Agent Blueprint
- Domain Service Blueprint
- Federated Integration Blueprint
- Implementation Blueprints
- Reference Deployment Blueprint
- Concept-to-Artifact Matrix
- Domain Dependency Matrix
- Reference Matrices
- Output and Authority Matrix
- GTOS Verification Matrix
- Correction Propagation Pattern
- Decision Recording Pattern
- Event Evolution Pattern
- Evidence Chain Pattern
- Exception and Case Management Pattern
- Federated Truth Pattern
- Identity Resolution Pattern
- GTOS Reference Patterns
- Long-Running Trade Workflow Pattern
- Relationship-Aware Access Pattern
- Temporal Reconstruction Pattern
- Scenario — Customs Declaration and Release
- End-to-End Trade Scenarios
- Scenario — Multi-Party Shipment Consolidation
- Scenario — Pharmaceutical Cold Chain
- Scenario — Sanctions and Restricted-Party Screening
- Scenario — Trade Finance and Settlement
- GTOS API Contract Standard
- Conformance Assessment Standard
- GTOS Event Contract Standard
- Financial Control Standard
- Cross-Cutting Standards
- Jurisdiction Model
- Semantic Migration Standard

- End-to-End Observability Standard
- GTOS Policy Standard
- Privacy and Purpose Standard
- GTOS Visual Architecture Atlas

ADR-0001 — Markdown Is the Canonical Source

Status

Accepted

Context

GTOS requires durable semantics and auditable architecture decisions across independently evolving technologies and organizations.

Decision

Versioned Markdown and registries are authoritative. Generated sites and exports are derivatives.

Consequences

- The decision becomes reviewable and testable.
- Implementations must preserve the declared semantic boundary.
- Exceptions require scope, owner, expiry, and review.
- Future replacement technology may vary while the architectural obligation remains.

Canon Traceability

CANON-001, CANON-002, and additional Canons applicable to the decision.

ADR-0002 — Retain Canon Terminology

Status

Accepted

Context

GTOS requires durable semantics and auditable architecture decisions across independently evolving technologies and organizations.

Decision

The branded term Canon denotes foundational axioms.

Consequences

- The decision becomes reviewable and testable.
- Implementations must preserve the declared semantic boundary.
- Exceptions require scope, owner, expiry, and review.
- Future replacement technology may vary while the architectural obligation remains.

Canon Traceability

CANON-001, CANON-002, and additional Canons applicable to the decision.

ADR-0003 — Ontology Precedes Event Architecture

Status

Accepted

Context

GTOS requires durable semantics and auditable architecture decisions across independently evolving technologies and organizations.

Decision

State and time are defined before Event because an Event evidences change.

Consequences

- The decision becomes reviewable and testable.
- Implementations must preserve the declared semantic boundary.
- Exceptions require scope, owner, expiry, and review.
- Future replacement technology may vary while the architectural obligation remains.

Canon Traceability

CANON-001, CANON-002, and additional Canons applicable to the decision.

ADR-0004 — Separate Ontology and Epistemology

Status

Accepted

Context

GTOS requires durable semantics and auditable architecture decisions across independently evolving technologies and organizations.

Decision

Reality and knowledge about reality are distinct architectural planes.

Consequences

- The decision becomes reviewable and testable.
- Implementations must preserve the declared semantic boundary.
- Exceptions require scope, owner, expiry, and review.
- Future replacement technology may vary while the architectural obligation remains.

Canon Traceability

CANON-001, CANON-002, and additional Canons applicable to the decision.

ADR-0005 — Domain Ownership over Shared Database

Status

Accepted

Context

GTOS requires durable semantics and auditable architecture decisions across independently evolving technologies and organizations.

Decision

Domains own propositions; integration uses governed contracts.

Consequences

- The decision becomes reviewable and testable.
- Implementations must preserve the declared semantic boundary.
- Exceptions require scope, owner, expiry, and review.
- Future replacement technology may vary while the architectural obligation remains.

Canon Traceability

CANON-001, CANON-002, and additional Canons applicable to the decision.

ADR-0006 — Recommendations Are Not Decisions

Status

Accepted

Context

GTOS requires durable semantics and auditable architecture decisions across independently evolving technologies and organizations.

Decision

AI and analytics outputs remain non-binding unless an authorized decision occurs.

Consequences

- The decision becomes reviewable and testable.
- Implementations must preserve the declared semantic boundary.
- Exceptions require scope, owner, expiry, and review.
- Future replacement technology may vary while the architectural obligation remains.

Canon Traceability

CANON-001, CANON-002, and additional Canons applicable to the decision.

ADR-0007 — Intent Gates Execution

Status

Accepted

Context

GTOS requires durable semantics and auditable architecture decisions across independently evolving technologies and organizations.

Decision

Side effects require traceable authorized intent.

Consequences

- The decision becomes reviewable and testable.
- Implementations must preserve the declared semantic boundary.
- Exceptions require scope, owner, expiry, and review.
- Future replacement technology may vary while the architectural obligation remains.

Canon Traceability

CANON-001, CANON-002, and additional Canons applicable to the decision.

ADR-0008 — Append-Only Correction

Status

Accepted

Context

GTOS requires durable semantics and auditable architecture decisions across independently evolving technologies and organizations.

Decision

Corrections preserve prior assertions and add governed superseding facts.

Consequences

- The decision becomes reviewable and testable.
- Implementations must preserve the declared semantic boundary.
- Exceptions require scope, owner, expiry, and review.
- Future replacement technology may vary while the architectural obligation remains.

Canon Traceability

CANON-001, CANON-002, and additional Canons applicable to the decision.

ADR-0009 — Static Portable Publication

Status

Accepted

Context

GTOS requires durable semantics and auditable architecture decisions across independently evolving technologies and organizations.

Decision

The handbook ships as a dependency-free static site with relative links.

Consequences

- The decision becomes reviewable and testable.
- Implementations must preserve the declared semantic boundary.
- Exceptions require scope, owner, expiry, and review.
- Future replacement technology may vary while the architectural obligation remains.

Canon Traceability

CANON-001, CANON-002, and additional Canons applicable to the decision.

ADR-0010 — Machine-Readable Registries

Status

Accepted

Context

GTOS requires durable semantics and auditable architecture decisions across independently evolving technologies and organizations.

Decision

Ownership, traceability, contracts, and decisions are represented in governed registries.

Consequences

- The decision becomes reviewable and testable.
- Implementations must preserve the declared semantic boundary.
- Exceptions require scope, owner, expiry, and review.
- Future replacement technology may vary while the architectural obligation remains.

Canon Traceability

CANON-001, CANON-002, and additional Canons applicable to the decision.

Architecture Decision Records

ADRs preserve consequential decisions, context, alternatives, and effects.

Architecture Quality Gates

Gate 1 — Semantic Integrity

All business concepts use glossary definitions. Ownership, identity, time, state, and relationship semantics are explicit. Commands and Events are distinct.

Gate 2 — Authority and Decision

Decision classes identify authority, evidence, policy, escalation, appeal, and expected outcome. AI outputs are typed. Execution traces to Intent.

Gate 3 — Data and History

Authoritative and derived data are distinguishable. Temporal semantics, lineage, quality, retention, correction, and re-construction are testable.

Gate 4 — Security and Privacy

Identity, delegation, purpose, jurisdiction, classification, authorization, audit, and threat controls are complete.

Gate 5 — Resilience and Operations

SLO, RTO, RPO, degradation, replay, reconciliation, runbooks, incident evidence, and recovery exercises exist.

Gate 6 — Change Safety

Contracts, compatibility, migration, rollback, observability, and deprecation are defined.

A release failing a mandatory Gate SHALL not claim architectural conformance.

Architecture Review Checklist

- Canon dependencies identified
- Truth ownership explicit
- Identity and time semantics explicit
- State and Event semantics valid
- Observations separated from authoritative facts
- Decision authority and evidence defined
- Intent precedes execution
- Security, privacy, resilience, and operations included
- Migration and correction behavior defined
- Conformance evidence available

Conformance Levels

Level 1 — Semantic

Uses GTOS terminology and preserves foundational distinctions.

Level 2 — Architectural

Implements ownership, contracts, temporal semantics, authority, security, and evidence.

Level 3 — Operational

Demonstrates SLOs, recovery, incident evidence, policy enforcement, and continuing validation.

Level 4 — Federated

Interoperates across organizations while preserving authority and jurisdiction.

Implementation Evidence Catalogue

Architecture Evidence

- context and container diagrams;
- domain ownership map;
- command and event catalogues;
- state models and invariants;
- Canon traceability matrix;
- ADRs and exception records.

Engineering Evidence

- schema validation;
- contract test reports;
- policy tests;
- migration tests;
- security scans;
- dependency and artifact provenance;
- resilience tests.

Operational Evidence

- SLO reports;
- incident records;
- restoration tests;
- recovery reconciliation;
- access and decision audit;
- AI evaluation and drift reports;
- unresolved risk and exception register.

Evidence SHALL identify version, scope, owner, collection time, and integrity.

Normative Language

SHALL and SHALL NOT define mandatory conformance. SHOULD and SHOULD NOT require justification when not followed. MAY denotes permission.

Release Checklist

- Validation passes
- Registries and schemas are valid
- Traceability report generated
- Site builds with relative links
- Manifest includes all release files
- ZIP integrity tested
- Version and changelog updated
- No unresolved critical conformance failures

Definition of Ready

A work item is ready for implementation only when:

- the business outcome and affected subject are explicit;
- truth ownership is known;
- required identity and temporal semantics are defined;
- state, invariants, Command, Event, and Decision behavior are understood;
- contracts and acceptance criteria exist;
- security classification and authority are defined;
- AI involvement, if any, has an approved output type and mandate;
- migration and compatibility impact are known;
- observability and operational expectations are included;
- open questions have owners and dates.

A backlog item that says only “build API,” “add AI,” or “sync data” is not ready.

Vertical Slice Delivery Guide

Objective

A vertical slice proves one meaningful business outcome across user interaction, domain logic, contract, persistence, policy, observability, and operation. It is not a thin user-interface prototype connected to placeholder logic. The slice must demonstrate that the architecture can carry a real governed fact from observation or command through outcome and evidence.

Slice Selection

Choose a slice with a clear subject, one or two critical invariants, a known truth owner, a small number of partner dependencies, and an observable business outcome. Avoid selecting the easiest technical endpoint if it bypasses identity, time, state, authority, or correction. A useful early slice often includes one Command, one Decision or policy check, two or three Events, a projection, and an operational alert.

Required Design

The slice SHALL define the domain aggregate, identifiers, temporal fields, Command contract, Event contract, authorization context, failure behavior, idempotency, data classification, SLO, and recovery test. If AI is involved, the output type, evidence source, evaluation, and execution boundary are explicit.

Implementation Sequence

1. Write scenario examples and acceptance evidence.
2. Define state and invariants before data tables.
3. Publish contract drafts and consumer expectations.
4. Implement domain behavior with invariant tests.
5. Add persistence and Event publication through an outbox or equivalent reliable pattern.
6. Build the projection and user or partner interface.
7. Add policy, security, telemetry, and runbook.
8. Exercise retry, duplicate, late data, correction, and dependency failure.
9. Demonstrate the end-to-end trace and business outcome.

Review Questions

Can a domain expert explain every state transition? Is an Event a completed fact rather than a request? Can a repeated Command create a duplicate effect? What happens when the response is lost? Can an operator see why the slice is blocked? Can a historical decision be reviewed using knowledge available at the time?

Completion

A vertical slice is complete when it runs in a production-like environment, contract tests pass, observability exposes technical and business state, security controls are enforced, a runbook is exercised, and Canon traceability is inspectable.

Architecture-Aware Backlog Decomposition

Problem

Traditional backlogs split work by screen, API, database, or team. This creates horizontal layers that appear busy but delay proof of business correctness. GTOS decomposes work around business outcomes and governed changes.

Work Item Anatomy

A well-formed work item identifies subject, desired outcome, truth owner, triggering observation or Intent, state transition, invariant, decision or policy, Command, Event, evidence, consumer, security context, and operational acceptance. Not every item requires all elements, but omitted elements are consciously assessed.

Decomposition Patterns

Use lifecycle slices such as create, validate, approve, execute, correct, and close. Use risk slices such as normal path, insufficient evidence, policy denial, partner delay, correction, and recovery. Use authority slices when human approval, delegated authority, and automated decision have different controls. Use temporal slices when current-state and historical reconstruction require separate projections.

Enablers

Foundational enablers include identity resolution, schema registry, policy enforcement, Event transport, evidence store, observability, and deployment automation. Enablers SHALL have explicit consumers and acceptance outcomes; otherwise they become infrastructure programs detached from value.

Anti-Patterns

“Create microservice,” “add Kafka,” “build AI agent,” “sync customer data,” and “make dashboard” are not outcome-ready items. They hide semantics and encourage technical completion without operational usefulness.

Definition of Ready

The team can state what becomes true, who owns that truth, how the change is authorized, which evidence proves completion, how failure is visible, and how rollback or correction behaves. Dependencies have owners and contract expectations.

Definition of Done

The implemented change includes tests, contracts, security, telemetry, documentation, migration, runbook, and traceability. Product acceptance confirms the business outcome, not only that the interface responds.

Contract Testing Guide

Scope

Contract testing protects meaning exchanged through APIs, Commands, Events, data products, policy interfaces, and AI tools. Syntax validation is necessary but insufficient. A consumer may parse a payload while reaching the wrong business conclusion because units, temporal meaning, authority, or defaults changed.

Test Layers

Schema tests validate required fields, types, formats, and structural compatibility. Semantic tests validate examples, units, identifier scope, state meaning, temporal interpretation, and authority. Provider tests prove production behavior conforms to the contract. Consumer tests prove supported versions and expected conclusions. Negative tests cover unauthorized subjects, stale evidence, invalid transitions, unknown enum values, and prohibited fields.

Event Tests

Event tests verify past-tense semantics, immutable identity, subject, source, occurrence and recording time, schema version, replay, ordering scope, duplicate handling, and correction references. A consumer SHALL tolerate at-least-once delivery and explicit version evolution.

API Tests

API tests verify authentication, relationship-aware authorization, idempotency, concurrency, pagination order, error categories, deprecation headers, and temporal query semantics. Commands test rejection and conflict as first-class outcomes.

Data Product Tests

Data products test grain, uniqueness, point-in-time correctness, lineage, quality thresholds, access classification, and backward compatibility. Analytical output SHALL not silently become authoritative operational truth.

Certification

External partners receive a conformance pack containing schemas, examples, test endpoints or fixtures, security requirements, replay and retry behavior, version policy, and support contacts. Certification results are versioned and expire when a breaking change or trust change occurs.

Release Gate

A release fails when material consumers are unknown, semantic changes lack migration, or contract tests do not cover authority and time. Temporary waivers identify affected consumers and sunset dates.

Production Readiness Review

Purpose

The Production Readiness Review establishes that a capability can safely carry real trade decisions and effects. It combines architecture, engineering, security, data, AI, and operations evidence.

Service Ownership

The service has an accountable product owner, domain owner, engineering owner, security contact, and on-call rotation. Escalation paths include partner, regulatory, financial, and executive contacts where applicable.

Functional Integrity

State models and invariants are implemented and tested. Commands are idempotent. Events represent completed facts. Corrections preserve history. Domain and contract versions are registered. Data migration and reconciliation are complete.

Security and Privacy

Threat models are current. Identities, credentials, delegation, policy, purpose, classification, secrets, encryption, audit, and incident containment are tested. Administrative capabilities use stronger controls. AI tools cannot bypass external policy enforcement.

Reliability

SLOs and error budgets reflect business journeys. Capacity covers peak, replay, recovery, and partner delay. Dependencies have timeout, retry, circuit, degradation, and ownership. Backups and restoration have been tested. Recovery includes business reconciliation.

Observability

Dashboards show availability, latency, errors, saturation, Event freshness, queue age, business state age, policy denial, evidence gaps, decision outcomes, and correction backlog. Alerts map to runbooks and accountable responders.

Change and Rollback

Deployment is staged and observable. Rollback criteria are objective. Rollback SHALL not delete new business facts or re-enable incompatible contracts. Feature flags, policy switches, tool disablement, or traffic controls support containment.

Go/No-Go

Critical findings block launch. Major findings require approved compensating controls and dates. The decision record captures attendees, evidence, dissent, risks, authority, and conditions. A successful review is valid only for the declared release, environment, and scope.

Cutover Command Center Guide

Objective

Cutover transfers operational responsibility, consumers, data, and business effects from an old arrangement to a new one while preserving identity, history, authority, and continuity.

Command Structure

The Cutover Commander coordinates decisions. Domain leads validate business truth. Data leads reconcile migration. Platform and infrastructure leads manage deployment. Security monitors access and threats. Operations controls incidents. Partner leads confirm external readiness. An evidence recorder maintains the authoritative timeline.

Readiness Board

The board tracks migration status, consumer status, contract versions, partner certification, data reconciliation, open defects, security state, SLO health, capacity, rollback window, and decision authority. Status must be based on current evidence, not verbal optimism.

Execution Plan

The plan defines freeze points, final extraction, transformation, load, validation, ownership switch, traffic shift, Event subscription position, workflow transfer, and communication. Each step has owner, expected duration, verification, and abort condition.

Data Reconciliation

Reconciliation compares business propositions and totals across identity, state, obligations, money, and history. Counts alone are inadequate. Sampling includes corrected records, late Events, disputed identities, and historical queries.

Rollback

Rollback distinguishes technical rollback from business reversal. Once new facts have been created, returning software does not make those facts disappear. The plan may require forward correction, compensation, dual read, or controlled pause rather than restoration of an old snapshot.

Closure

Cutover closes after stability criteria, consumer confirmation, reconciliation, incident review, and decommission actions are assigned. Elevated monitoring continues for a defined period. Temporary credentials, bridges, and exceptions are removed or governed.

Partner Certification Guide

Purpose

Partner certification verifies that an external organization can exchange governed facts and requests safely, not merely connect technically.

Identity and Relationship

Certification begins with verified organizational identity, legal or contractual relationship, technical identities, credentials, purposes, jurisdictions, data scopes, authority, support contacts, and offboarding terms. Partner codes are linked to stable Party identities rather than used as universal identity.

Contract Profile

The profile lists supported API, Command, Event, document, and bulk versions; optional fields; classification; time semantics; retry; ordering; signature or integrity; rate and volume; retention; and deprecation contacts.

Test Scenarios

Tests include valid exchange, invalid schema, unauthorized subject, expired mandate, duplicate Command, replayed Event, late Event, corrected fact, unavailable dependency, partial batch, and credential revocation. High-impact domains include business reconciliation and negative authority cases.

Operational Certification

Partners demonstrate monitoring, incident contact, maintenance communication, queue or retry handling, and recovery. GTOS verifies that partner downtime does not create invented approval or uncontrolled execution.

Security

Certification assesses credential protection, network or channel controls, encryption, least privilege, audit, vulnerability management, and incident notification. A trust change may require re-certification.

Evidence and Status

Certification records scope, version, environment, tests, findings, exceptions, approval, expiry, and next review. Certification does not grant broader access than the declared Relationship and purpose. Contract changes, acquisitions, jurisdiction changes, or security incidents can suspend status.

Observability Dashboard Design

Audience Views

Executives need value-stream outcomes, risk, and benefit realization. Product and domain owners need flow, exceptions, decisions, and data quality. Engineers need service and dependency behavior. Security needs identity, policy, access, and threat signals. Operations needs actionable health and runbook context.

Dashboard Layers

The first layer shows business journey success, volume, latency, blocked work, and SLO status. The second shows domain state, Command rejection, Event freshness, evidence gaps, decision outcomes, and correction backlog. The third shows technical resources, dependencies, releases, queues, and error detail.

Required Dimensions

Metrics SHOULD support domain, tenant or participant, jurisdiction, version, environment, partner, value stream, decision type, and severity where classification permits. High-cardinality identifiers are searched through protected logs or traces rather than placed indiscriminately in metric labels.

Causal Navigation

A user moves from an outcome metric to affected subjects, correlated workflows, policy evaluations, Events, traces, and deployment changes. Correlation assists investigation but does not prove causality.

Alert Design

Alerts indicate owned impact: “customs release events are stale for EU imports” is stronger than “consumer lag above threshold.” Every alert has severity, user impact, owner, runbook, suppression rule, and completion signal.

AI and Decision Monitoring

Dashboards show model route, evaluation status, evidence faithfulness, escalation, decision reversal, unsafe tool attempts, drift, and containment state. Quality decline can suspend automation before general service failure.

Review

Dashboards are reviewed in production readiness and after incidents. Unused panels are removed. Missing signals become engineering work with owners and deadlines.

Security Verification Guide

Verification Strategy

GTOS security verification combines design review, automated tests, abuse cases, adversarial exercises, penetration testing, policy simulation, access review, and incident drills. It covers business authority as well as technical exploitation.

Identity Tests

Verify human, workload, partner, device, and Agent identity lifecycle. Test expired, revoked, duplicated, and compromised credentials. Confirm that authentication alone does not grant domain authority.

Authorization Tests

Test positive and negative cases across role, Relationship, delegation, purpose, jurisdiction, classification, time, and risk. Confirm cached authorization expires no later than mandate validity. Test object-level and field-level access.

Business Abuse Cases

Attempt duplicate settlement, unauthorized release, forged Event, replayed Command, ownership takeover, document substitution, evidence deletion, identifier merge abuse, and emergency-access misuse. Verify invariant and audit controls.

AI Adversarial Tests

Test prompt injection, poisoned evidence, tool argument manipulation, secret extraction, cross-tenant context, mandate expansion, unsafe chaining, fabricated approval, and denial of wallet. Tool gateways SHALL deny unauthorized actions independent of model output.

Resilience and Incident

Test credential revocation during active workflow, partner compromise, key rotation, audit-store degradation, policy-service failure, and emergency containment. Verify evidence remains available and response does not produce unauthorized default permit.

Release Evidence

Findings include severity, exploit path, business impact, affected authority, evidence, remediation, owner, and verification. Critical findings block release. Approved residual risk has accountable authority and expiry.

Data Pipeline Engineering Guide

Source Classification

Every source is classified as authoritative, observational, derived, reference, or external unverified. The pipeline preserves source identity, contract version, extraction time, event or observation time, and transformation lineage.

Ingestion

Ingestion validates schema, integrity, authority, subject identity, classification, and temporal fields. Invalid records are quarantined with reason and ownership. The system does not silently coerce identifiers, currencies, units, or unknown enum values.

Transformation

Transformations are deterministic where practical, versioned, tested, and traceable. Normalization preserves original representation. Joins declare temporal semantics and identity rules. AI extraction or classification creates derived claims with confidence and source span.

Quality

Quality checks cover completeness, validity, uniqueness, consistency, timeliness, authority, and lineage. Thresholds are tied to use; a dataset suitable for trend analysis may be unsafe for a legal decision.

Publication

A data product publishes grain, owner, schema, metric definitions, quality SLOs, access policy, retention, residency, and consumer support. Projections clearly disclose freshness and correction behavior.

Replay and Backfill

Backfill is governed because current transformation logic may reinterpret historical data. The pipeline records code and rule versions and separates historically reproduced results from newly derived perspectives.

Operations

Operators monitor lag, quarantine, schema change, quality drift, lineage failure, correction propagation, cost, and consumer impact. Recovery verifies completeness and avoids duplicate downstream effects.

AI Release Engineering Guide

Release Unit

An AI release includes model, route, prompt or task definition, retrieval configuration, tool registry, policy, structured output schema, evaluation set, thresholds, observability, and rollback controls. Changing any component can change behavior.

Pre-Release Evaluation

Evaluate task quality, factuality, evidence faithfulness, calibration, uncertainty, prohibited behavior, bias, security, latency, cost, and tool safety. High-impact use cases include representative edge cases, contradictory evidence, missing evidence, policy changes, and adversarial inputs.

Staged Deployment

Use offline evaluation, shadow operation, limited internal users, low-impact traffic, controlled participant cohorts, and progressive automation. Compare recommendations or decisions against an approved baseline. Do not expose execution tools before recommendation and policy behavior are stable.

Runtime Controls

The runtime validates structured output, enforces policy outside the model, limits tools and egress, applies budgets, records evidence and tool calls, and supports human escalation. A kill switch can disable model routes, tools, Mandates, or decision classes.

Monitoring

Track success, evidence support, hallucination, unsafe requests, tool denial, escalation, decision reversal, downstream outcome, latency, and cost. Drift includes data, policy, prompt, tool, and user behavior—not only model statistics.

Rollback

Rollback returns to an approved release configuration. It does not rewrite decisions or outcomes already produced. A harmful decision may require appeal, correction, notification, or remediation.

Approval

AI owner, domain authority, security, data, operations, and architecture approve according to impact. The Decision Record identifies evidence and conditions for continued operation.

Performance and Capacity Engineering

Business Workload Model

Capacity begins with trade volumes, lifecycle peaks, partner deadlines, jurisdictional cutoffs, document size, Event fan-out, workflow duration, analytical demand, and AI usage. Average requests per second alone is insufficient.

Critical Paths

Identify synchronous decision paths, asynchronous flow paths, batch deadlines, recovery replay, and user investigation. Each has latency and throughput objectives. External authorities may dominate latency; the system must expose blocked dependency rather than appear generally slow.

Load Profiles

Test normal, peak, burst, partner reconnect, backlog drain, Event replay, migration, incident diagnostics, and regional failover. Include large Relationships, long histories, high-cardinality searches, and worst-case policy evaluation.

Resource Controls

Apply quotas, fairness, backpressure, rate limits, queue isolation, workload priority, and AI token or tool budgets. One tenant, partner, Agent, or replay SHALL not exhaust shared capacity.

Degradation

Define which functions can serve stale projections, queue work, require manual review, or suspend action. Degradation SHALL not bypass authorization, evidence, or invariant checks.

Cost

Track unit cost by business outcome, domain, partner, data product, and AI use case. Cost controls SHALL not delete required history, reduce audit, disable security, or route sensitive data through unapproved processors.

Capacity Review

Review before major seasonal peaks, partner launches, policy changes, model expansion, or migrations. Evidence includes forecasts, tests, observed headroom, dependency limits, and contingency.

Post-Launch Stabilization Guide

Stabilization Window

A defined stabilization period follows launch or major migration. Change volume is controlled, monitoring is heightened, and the delivery team remains connected to operations and domain owners.

Daily Review

Review SLOs, business completion, Command rejection, Event lag, data reconciliation, policy denial, partner incidents, evidence gaps, security signals, AI outcomes, user feedback, and support cases. Metrics are compared with launch assumptions.

Defect Classification

Distinguish semantic defects, contract defects, data defects, policy defects, security defects, operational defects, usability issues, and training gaps. A symptom such as “wrong status” may originate in ownership, temporal projection, late Event, or UI language.

Decision Discipline

Urgent fixes still record authority and evidence. Temporary bypasses have scope, owner, expiry, and detection. Stabilization SHALL not normalize manual edits to authoritative stores.

Reconciliation

High-impact domains perform daily or continuous reconciliation of identities, obligations, shipments, decisions, settlements, and Event positions. Differences are investigated through lineage rather than overwritten.

Transition to Normal Operations

Exit criteria include sustained SLOs, reconciled data, manageable incident and defect rate, trained support, closed critical risks, stable partner behavior, verified runbooks, and accepted ownership. Remaining work transfers to a visible backlog.

Learning

The team updates architecture patterns, tests, capacity assumptions, onboarding material, and delivery gates based on observed behavior. Benefits are measured against the business baseline.

Ninety-Day Domain Onboarding Roadmap

Days 1-15 — Establish Meaning

Appoint the Domain Owner, Product Manager, Architect, Data Steward, Engineering Lead, Security partner, and Operations partner. Define purpose, non-goals, critical business outcomes, participating actors, jurisdictions, and current systems. Run the Truth Ownership Workshop and identify the propositions for which the domain will be authoritative.

Build the ubiquitous language from real documents, incidents, exceptions, and decisions. Record ambiguous terms rather than selecting convenient definitions prematurely. Identify stable Entities and identifiers, business Relationships, temporal dimensions, state, and history requirements.

Days 16-30 — Model Behavior

Create aggregate candidates, invariants, lifecycle diagrams, Commands, Events, Observations, Decisions, policies, and exception cases. Test the model through at least three normal scenarios and five failure scenarios: duplicate, late information, contradiction, correction, and unauthorized action.

Publish a domain charter and preliminary contract catalogue. Identify upstream authorities and downstream consumers. Decide which consistency rules remain inside the aggregate and which cross-domain outcomes require workflow and compensation.

Days 31-50 — Design Contracts and Controls

Draft API, Command, Event, and data contracts. Define temporal queries, idempotency, versioning, errors, replay, correction, data quality, retention, classification, residency, and lineage. Create the threat model and authorization model, including partner Relationships and Agent mandates.

Set initial SLOs and operational indicators. Define evidence required for critical Decisions and historical reconstruction. Review the design against all applicable Canons and constitutional articles.

Days 51-75 — Build Vertical Slices

Implement one or two vertical slices using production engineering standards. Add invariant, contract, policy, security, migration, resilience, and operational tests. Create dashboards and runbooks while building, not after functional completion.

Certify at least one realistic consumer or partner simulator. Demonstrate duplicate handling, dependency failure, correction propagation, and projection rebuild.

Days 76-90 — Launch Readiness

Complete migration and reconciliation, performance tests, security verification, recovery exercise, production readiness, and ownership transfer. Launch through controlled cohorts. Measure business outcomes and exception behavior. Record architecture debt and the next roadmap.

At day ninety the domain need not be feature-complete, but it SHALL have credible authority, contracts, operations, and a repeatable delivery model.

Architecture Design Review Facilitation

Before the Review

The author distributes a concise context, proposed decision, diagrams, scenarios, Canon dependencies, alternatives, risks, contracts, and open questions. Reviewers receive enough time to inspect the material. A meeting is not used to reveal an unread design for the first time.

Review Sequence

Begin with business reality and intended outcome. Confirm the truth owner and authority boundary. Walk through identities, Relationships, time, state, Commands, Events, Decisions, Intent, and corrections. Then review data, AI, security, platform, infrastructure, and operations.

Use concrete scenarios rather than debating abstract preferences. Include a normal case, an invalid transition, late evidence, contradictory evidence, retry after response loss, partner outage, security abuse, and disaster recovery.

Decision Matrix

Dimension	Questions
Correctness	Does the design preserve the GTOS concepts and invariants?
Authority	Are assertions, Decisions, delegation, and execution legitimate?
Compatibility	Can existing consumers and historical data retain meaning?
Security	Are trust boundaries, purpose, jurisdiction, and abuse controlled?
Operability	Can teams observe, support, recover, and reconcile?
Evolution	Can the design change without semantic fragmentation?
Cost	Is complexity proportional to business impact?

Review Outcomes

Outcomes are approved, approved with conditions, revise and return, or rejected. Conditions have owners and due dates. Dissent and rejected alternatives are preserved. Approval applies to a version and scope, not to all future implementation changes.

Healthy Review Behavior

Reviewers challenge the design, not the person. Seniority does not replace evidence. Novelty is neither a virtue nor a defect. A simpler design wins only when it preserves required semantics and controls. A sophisticated design is rejected when operational ownership is not credible.

After the Review

Update the ADR, architecture models, risks, contracts, and backlog. Verify that conditions enter delivery planning. Significant deviations return to review rather than appearing only in code.

Command Handler Reference Flow

Input

A Command arrives with command identity, type, subject, initiator, authority or Intent reference, requested time, idempotency key, expected version where used, schema version, and payload.

Processing Steps

1. Authenticate the technical caller and resolve the acting identity.
2. Validate schema and supported version.
3. Retrieve or validate the authority, delegation, mandate, and purpose.
4. Check the idempotency store. If the logical Command completed earlier, return the recorded outcome rather than execute again.
5. Resolve the subject identity and load authoritative state under the aggregate consistency boundary.
6. Evaluate policy and required evidence.
7. Validate state transition and invariants.
8. Apply domain behavior and produce new state plus one or more Events.
9. Persist state and outgoing Events atomically through an outbox, event store, or equivalent guarantee.
10. Record the outcome, decision or policy references, correlation, and telemetry.

Result Types

The handler returns accepted and completed, accepted for asynchronous processing, rejected by validation, denied by policy, conflict with current state, insufficient evidence, duplicate with prior outcome, or temporarily unavailable. A generic failure response hides business meaning and complicates safe retry.

Concurrency

Optimistic concurrency is appropriate when conflicts are expected to be rare and domain rules can reject stale commands. Pessimistic coordination MAY be required for high-contention financial or inventory invariants. Distributed locks SHALL not become invisible ownership.

Failure After Commit

If state commits but the response is lost, a retry with the same idempotency key returns the previous result. If Event publication is delayed, the outbox is retried. The handler SHALL not apply the business transition a second time.

Audit and Observability

Record Command outcome, policy result, invariant conflict, processing latency, Event publication, and dependency failures. Sensitive payloads are minimized in logs. Operators can locate the full trace through identifiers and evidence references.

Event Consumer Reference Flow

Receipt

The consumer receives an Event envelope and records transport receipt time. Receipt is not acceptance. The consumer verifies source identity, integrity, schema support, subject, classification, and authorization to process the Event.

Deduplication

Event identity is checked against the consumer's processed-event record or idempotent business operation. A duplicate may update transport metrics but SHALL not repeat business effects.

Interpretation

The consumer applies semantics for the declared Event version. Occurrence time and recording time remain distinct. Ordering assumptions are limited to the producer's documented scope. An Event arriving late may still change a projection or trigger an impact assessment.

Processing Types

Projection consumers update a derived view. Workflow consumers advance coordination state. Domain consumers may issue a new Command under their own authority and policy; they do not directly mutate the producer's truth. Notification consumers communicate under replay controls. Analytical consumers preserve lineage.

Failure

Transient infrastructure failure triggers bounded retry with backoff. Invalid schema, unsupported semantic version, prohibited data, or invariant contradiction enters quarantine or a case with owner and reason. Infinite retry without visibility is prohibited.

Replay

Replay is explicitly authorized. Consumers know whether they are rebuilding state, re-evaluating current policy, or reproducing historical behavior. Irreversible notifications and payments are disabled unless replay Intent specifically permits them.

Correction

A correction Event references the original. The consumer identifies affected projections, Decisions, documents, or executions. Some results are recomputed; others remain historically valid and create remediation. The correction is never treated as proof that the original Event did not exist.

Operational Signals

Monitor lag by occurrence and receipt, processing success, quarantine, duplicates, unsupported versions, replay position, correction backlog, and business outcome. Queue depth alone is not enough.

Business Recovery Exercise

Objective

The exercise proves that GTOS can restore service and reconcile business truth after a realistic failure. A successful infrastructure failover without business completeness is a failed exercise.

Scenario Design

Select a scenario that crosses domains and trust boundaries: regional outage during customs release, Event transport loss during shipment custody transfer, compromised partner credential during payment processing, or data-store restoration while corrections are in flight.

Define failure time, affected components, data boundary, known and hidden symptoms, expected detection, business deadlines, and legal or partner obligations. Inject ambiguity so responders must acquire evidence rather than follow a scripted answer.

Exercise Phases

1. Detect and declare the incident.
2. Establish command roles and preserve evidence.
3. Contain unsafe execution and compromised authority.
4. Restore identity, policy, authoritative stores, Event transport, workflow, and projections in dependency order.
5. Identify gaps, duplicates, stale projections, and unresolved Commands.
6. Reconcile identities, state, obligations, Decisions, Intent, Events, and financial totals.
7. Notify participants and regulators where required.
8. Return to controlled operation and close temporary authority.

Success Criteria

RTO and RPO are met or variances explained. No unauthorized default action occurs. Historical and current state can be reconstructed. Duplicate effects are prevented or corrected. Partner exchanges are reconciled. Evidence proves each restoration and decision.

Review

The review distinguishes technical restoration, business reconciliation, and stakeholder recovery. Actions address architecture, tooling, runbooks, access, training, capacity, and partner agreements. The exercise is repeated until critical weaknesses are verified closed.

Architecture Documentation Standard

Purpose

GTOS documentation is part of the control system. It communicates meaning, supports delivery, enables audit, and preserves decisions. Documents that cannot be maintained or verified create false confidence.

Required Metadata

Normative documents declare title, purpose, owner, status, version, scope, dependencies, review date, and approval authority. Generated pages inherit repository version and source path. Machine-readable registries are used when structured ownership or traceability is required.

Writing Rules

Use business terms from the glossary. Define new terms before use. Use SHALL only for objective conformance. Separate facts, assumptions, decisions, recommendations, examples, and open questions. Prefer tables for comparisons and matrices, diagrams for structure and flow, and scenarios for behavior.

Diagram Rules

Every diagram has purpose, scope, legend where needed, and owned source. Logical diagrams SHALL not imply physical deployment accidentally. Arrows state meaning: data, Command, Event, authority, or dependency. Colors are not the sole carrier of information.

Currency

Documentation changes with the implementation when meaning or operation changes. Broken links, obsolete owners, unsupported versions, and unresolved placeholders fail release validation. Generated site files are not edited directly.

Decision Records

Consequential decisions use ADRs with context, options, decision, consequences, Canon traceability, and review trigger. Chat transcripts and presentation slides MAY support context but are not substitutes for the approved record.

Operational Documentation

Runbooks contain triggers, authority, prerequisites, steps, evidence, rollback, escalation, and completion. During incidents, operators must be able to navigate from alert to runbook, dependency, owner, dashboard, and recent changes.

Accessibility

Tables have clear headers, diagrams include text alternatives, language is direct, pages print cleanly, and UI supports keyboard navigation and responsive layouts.

Architecture Debt Management

Definition

Architecture debt is a known divergence from the target semantics, controls, or operating model that increases future change cost or present risk. Debt is not every imperfect implementation. It is a governed condition with observable consequence.

Debt Record

A debt item identifies affected capability, Canon or standard, reason, owner, introduced date, business and technical impact, security or compliance exposure, dependent work, compensating control, remediation options, cost estimate, and expiry or review date.

Categories

Semantic debt includes ambiguous ownership, identifiers, state, or contracts. Data debt includes missing lineage, temporal meaning, or quality. Security debt includes weak delegation, purpose, or audit. Operational debt includes absent SLOs, runbooks, or recovery. Platform debt includes unsupported dependencies and brittle shared capabilities.

Prioritization

Prioritize by probability and impact of invalid business effect, blocked evolution, incident exposure, regulatory consequence, partner burden, and repeated exception cost. A small code cleanup is not automatically more important than an unowned proposition used in financial decisions.

Controls

Debt with critical impact may block release. Accepted debt has accountable authority and compensating controls. Repeated temporary exceptions are converted into debt and reviewed at portfolio level. Delivery metrics SHOULD track debt created and retired, not reward teams for hiding it.

Remediation

Remediation may involve semantic clarification, contract version, migration, policy, testing, platform capability, or organizational ownership. Completion requires evidence that the risk and dependent work are resolved, not only that a ticket closed.

GTOS Team Ceremonies

Weekly Domain Review

Review domain decisions, new terms, invariant changes, contract changes, corrections, and consumer issues. The Domain Owner and Architect ensure meaning remains coherent.

Contract Review

Producers and material consumers examine API, Event, and data evolution. The review focuses on semantic compatibility, time, authority, retry, and migration. Schema diffs are supporting evidence rather than the whole review.

Decision and AI Review

For automated or AI-assisted decision classes, review evidence quality, model and policy versions, escalation, reversal, appeals, outcome quality, drift, security signals, and containment readiness.

Operational Review

Review SLOs, error budgets, incidents, correction backlog, Event freshness, partner health, capacity, vulnerabilities, and runbook actions. Business and technical signals appear together.

Architecture Council

The council resolves cross-domain ownership, constitutional exceptions, major ADRs, shared standards, and debt. It does not micromanage local implementation choices that conform to the architecture.

Retrospective

Retrospectives identify conditions in process, tooling, architecture, and organization. Actions have owners and evidence. They do not rewrite the history of decisions or blame individuals for predictable system conditions.

Portfolio Review

Executives review value realization, adoption, foundation investment, domain roadmap, major risk, partner scale, and decommission progress. Teams demonstrate business outcomes and conformance rather than only feature counts.

Ceremony Discipline

Every recurring ceremony has purpose, inputs, decisions, outputs, participants, and cancellation criteria. Meetings without owned outcomes are removed. Decisions are captured in durable artifacts rather than remaining in conversation.

Quality Assurance Strategy

Quality Model

GTOS quality is the demonstrated preservation of business meaning, authority, history, security, and operational outcomes. A system can have excellent code coverage and still be low quality if it produces invalid business effects or cannot explain Decisions.

Assurance Layers

Requirements assurance checks that outcomes, ownership, terminology, and constraints are understood. Model assurance checks state, invariants, Relationships, temporal semantics, and corrections. Contract assurance checks APIs, Events, data products, and policies. Implementation assurance checks code, dependencies, migration, security, and performance. Operational assurance checks SLOs, incidents, recovery, reconciliation, and continuing evidence.

Scenario Portfolio

Every critical capability maintains scenario sets covering normal flow, boundary conditions, invalid transition, missing evidence, contradictory evidence, late observation, duplicate Command, duplicate Event, out-of-order Event, partner outage, policy change, correction, replay, security abuse, recovery, and manual intervention.

Test Data

Test data represents realistic identifiers, currencies, units, time zones, jurisdictions, document versions, Relationships, and historical sequences. Synthetic data must preserve the structural difficulty of production without exposing protected information. Test fixtures include authority and provenance, not only payload fields.

Independence

High-impact controls receive independent review or verification. The same implementation team MAY write automated tests, but decision automation, payment, release, identity merge, and security boundaries SHOULD also be challenged by separate domain, risk, or security reviewers.

Evidence

Quality evidence is versioned and linked to release artifacts. It includes test results, coverage of invariants and scenarios, unresolved defects, accepted risks, environment, dataset or fixture versions, and reviewer identity. A passing dashboard without reproducible evidence is insufficient.

Continuous Improvement

Production incidents, appeals, corrections, partner failures, and near misses become new scenarios and controls. Quality metrics track escaped semantic defects, decision reversals, correction impact, contract incidents, recovery gaps, and automation suspension—not only defect counts.

GTOS Project Plan Template

Workstreams

A GTOS project plan normally includes business architecture, domain design, contracts and integration, data and migration, AI and decision intelligence, security and privacy, platform and infrastructure, engineering enablement, partner onboarding, testing, operations, change management, and benefit realization.

Milestones

Milestones align to delivery stage gates rather than arbitrary percentage completion. Recommended milestones are mandate approval, Reference Alignment, domain ownership approval, contract baseline, operational design approval, first vertical slice, partner certification, migration rehearsal, production readiness, controlled launch, and stabilization exit.

Dependency Management

Dependencies identify provider, consumer, required artifact, version, due date, acceptance evidence, and fallback. “Platform dependency” is too vague. The plan states whether the dependency is an identity capability, policy contract, Event topic, data product, environment, partner decision, or operational readiness item.

Critical Path

The critical path often passes through semantic and organizational decisions rather than coding. Unresolved truth ownership, authority, partner mandate, historical migration, or jurisdiction policy can block safe implementation. Project reporting SHALL expose these conditions rather than presenting engineering activity as equivalent progress.

Status Reporting

Status includes business outcome, stage-gate evidence, decisions required, high risks, architecture debt, contract readiness, migration reconciliation, security findings, SLO readiness, partner status, and benefit measures. Red status is used when a critical semantic or authority decision is absent even if code delivery is on schedule.

Change Control

Scope changes are assessed for Canon, domain, contract, data, AI, security, migration, and operations impact. A user-interface request may still change authority or state semantics. Changes with breaking meaning return to architecture review.

Closure

Project closure transfers products, ownership, runbooks, contracts, registries, risk, debt, support, and benefit tracking into normal portfolio operations. Temporary environments, credentials, integrations, and exceptions are retired or governed.

Engineer Onboarding Guide

First Principles

A new GTOS engineer first learns the conceptual distinctions that drive implementation: reality is not the database; identity is not an identifier; state is scoped; Events are completed facts; Observations are not automatically truth; recommendations are not Decisions; execution requires Intent; correction preserves history.

First Week

Read the Reference Theory, Canon, Constitution, Domain Architecture, Engineering Standards, and the charter for the assigned domain. Review one end-to-end scenario, one incident, one ADR, one Command, one Event, and one runbook. Set up the local environment and run validation and build tools.

First Assignment

The initial assignment SHOULD be a bounded vertical slice or defect that crosses model, test, observability, and documentation. Purely cosmetic work does not teach authority, contracts, or operations. Pair with a domain expert and experienced engineer.

Required Competencies

The engineer can explain the domain owner, aggregates, invariants, Command and Event semantics, temporal fields, idempotency, contract versions, authorization context, data classification, SLOs, and recovery. AI engineers additionally explain evidence, output type, Mandate, tools, evaluation, and containment.

Development Workflow

Before coding, confirm Definition of Ready. During implementation, update tests, contracts, documentation, telemetry, and ADRs. Before review, run validation and demonstrate failure paths. Code review examines business meaning and operability, not only style.

Production Responsibility

Engineers participate in production readiness, observe deployments, learn runbooks, and review incidents. Ownership does not end when code merges. Teams are expected to understand how their capability behaves under duplicate, delay, correction, dependency failure, and recovery.

Learning Path

After domain proficiency, engineers rotate through partner integration, security or policy, data lineage, AI evaluation, and operations. The objective is not that every engineer becomes a chief architect, but that local decisions preserve the end-to-end operating model.

Engineering Acceptance and Handover

Purpose

Engineering acceptance confirms that a GTOS capability is not merely implemented but ready to become an owned, supportable, auditable part of the operating system. Acceptance transfers responsibility from a temporary delivery arrangement to durable product and operations ownership.

Acceptance Package

The package includes approved domain and solution design, Canon traceability, ADRs, contracts, schemas, source repositories, build and deployment definitions, dependency inventory, test evidence, threat model, data classification, migration and reconciliation evidence, dashboards, SLOs, alerts, runbooks, support rota, known risks, architecture debt, partner contacts, and recovery results.

Knowledge Transfer

Handover combines documentation with practical demonstration. Receiving engineers deploy the capability, diagnose a simulated failure, trace a business outcome, execute a runbook, inspect a Decision Record, replay or rebuild a projection safely, and explain correction behavior. Attendance at a presentation does not prove operational competence.

Ownership Transfer

The Product Owner accepts outcome and roadmap. The Domain Owner accepts authoritative meaning and policy. Engineering accepts code and delivery pipelines. Data accepts contracts, lineage, quality, retention, and access. Security accepts controls and residual risk. Operations accepts SLOs, runbooks, on-call, recovery, and incident escalation.

Open Items

Open defects, exceptions, debt, manual controls, temporary credentials, feature flags, migration bridges, and unsupported consumers remain visible with owner, impact, due date, and escalation. Hidden unfinished work invalidates acceptance.

Acceptance Decision

The acceptance Decision identifies scope, release, environment, evidence, conditions, dissent, and effective date. Conditional acceptance has measurable exit criteria. Critical non-conformance cannot be converted into acceptance merely because a project deadline has arrived.

Post-Acceptance Verification

Within the agreed period, the receiving team reviews production outcomes, SLOs, incidents, partner behavior, data reconciliation, AI quality, costs, and benefit measures. Material divergence triggers remediation or a new architecture Decision. Temporary project access and elevated permissions are removed after verified transfer.

Final Verification Questions

Can the receiving team identify every production owner and escalation path? Can it explain which data is authoritative and which is derived? Can it safely stop automated execution without losing evidence? Can it recover from backup,

replay Events, reconcile business state, and prove completion? Can it trace a user-visible outcome to policy, Decision, Intent, Command, Event, deployment, and operational telemetry? Can it identify all temporary controls and their expiry? If any answer is unknown, the handover remains incomplete and the delivery team retains responsibility for closing the gap.

Acceptance evidence SHALL remain accessible, versioned, attributable, and reviewable throughout the operational life of the capability.

Engineering Execution Guides

These guides provide concrete implementation and launch practices for project engineers and delivery teams.

GTOS Delivery Handbook

The Delivery Handbook is written for program leaders, project managers, product owners, architects, engineers, analysts, security teams, data teams, AI teams, and operations. It converts the architecture into stage gates, work products, review questions, and completion evidence.

A stage is complete only when the required architectural evidence exists. Presentation decks without inspectable models, contracts, tests, and accountable ownership do not satisfy the gate.

Playbook — AI Use-Case Qualification

Purpose

Determine whether AI is appropriate and what authority boundary applies.

When to Use

Use this playbook when creating or materially changing a GTOS domain, platform capability, integration, data product, AI capability, security control, migration, or operational process. The facilitator SHALL adapt detail to impact without removing foundational questions.

Preparation

Identify sponsor, accountable owner, participants, decisions required, source evidence, current constraints, affected jurisdictions, and the deadline. Distribute existing domain models, contracts, incidents, policies, and metrics before the session.

Working Steps

1. Classify output type.
2. Identify evidence and uncertainty.
3. Assess impact and reversibility.
4. Define human oversight.
5. Select evaluation metrics.
6. Design containment.

Facilitation Questions

- What exists in business reality independently of our software?
- Who owns each proposition and under what authority?
- Which identifiers remain stable through change?
- Which time dimensions matter?
- What state and invariants must be protected?
- Which messages are Commands, Events, Observations, Recommendations, or Decisions?
- What evidence is available and what remains uncertain?
- What Intent authorizes execution?
- How are correction, replay, recovery, and dispute handled?

Outputs

- use-case card
- risk classification
- agent mandate
- evaluation plan
- tool policy
- incident controls

Quality Controls

Outputs SHALL use GTOS terminology, identify Canon dependencies, distinguish authority from technical ownership, and include unresolved questions. Decisions made during the playbook are captured in ADRs or governed registries.

Completion Evidence

The playbook is complete when accountable owners approve the outputs, downstream teams can implement from them, testable acceptance criteria exist, and critical uncertainty is either resolved or formally controlled.

Playbook — Architecture Decision Review

Purpose

Make consequential design choices transparent and reversible where possible.

When to Use

Use this playbook when creating or materially changing a GTOS domain, platform capability, integration, data product, AI capability, security control, migration, or operational process. The facilitator SHALL adapt detail to impact without removing foundational questions.

Preparation

Identify sponsor, accountable owner, participants, decisions required, source evidence, current constraints, affected jurisdictions, and the deadline. Distribute existing domain models, contracts, incidents, policies, and metrics before the session.

Working Steps

1. State context and constraints.
2. Identify canon dependencies.
3. Compare options.
4. Analyze migration and failure.
5. Record dissent.
6. Define review trigger.

Facilitation Questions

- What exists in business reality independently of our software?
- Who owns each proposition and under what authority?
- Which identifiers remain stable through change?
- Which time dimensions matter?
- What state and invariants must be protected?
- Which messages are Commands, Events, Observations, Recommendations, or Decisions?
- What evidence is available and what remains uncertain?
- What Intent authorizes execution?
- How are correction, replay, recovery, and dispute handled?

Outputs

- ADR
- decision matrix
- risk and consequences
- review date

Quality Controls

Outputs SHALL use GTOS terminology, identify Canon dependencies, distinguish authority from technical ownership, and include unresolved questions. Decisions made during the playbook are captured in ADRs or governed registries.

Completion Evidence

The playbook is complete when accountable owners approve the outputs, downstream teams can implement from them, testable acceptance criteria exist, and critical uncertainty is either resolved or formally controlled.

Playbook — Data Product Onboarding

Purpose

Publish governed operational or analytical data for reuse.

When to Use

Use this playbook when creating or materially changing a GTOS domain, platform capability, integration, data product, AI capability, security control, migration, or operational process. The facilitator SHALL adapt detail to impact without removing foundational questions.

Preparation

Identify sponsor, accountable owner, participants, decisions required, source evidence, current constraints, affected jurisdictions, and the deadline. Distribute existing domain models, contracts, incidents, policies, and metrics before the session.

Working Steps

1. Declare owner and consumers.
2. Define grain and semantics.
3. Publish schema and quality rules.
4. Add lineage and classification.
5. Test point-in-time correctness.
6. Define deprecation.

Facilitation Questions

- What exists in business reality independently of our software?
- Who owns each proposition and under what authority?
- Which identifiers remain stable through change?
- Which time dimensions matter?
- What state and invariants must be protected?
- Which messages are Commands, Events, Observations, Recommendations, or Decisions?
- What evidence is available and what remains uncertain?
- What Intent authorizes execution?
- How are correction, replay, recovery, and dispute handled?

Outputs

- data contract
- quality SLOs
- lineage graph
- access policy
- consumer registry

Quality Controls

Outputs SHALL use GTOS terminology, identify Canon dependencies, distinguish authority from technical ownership, and include unresolved questions. Decisions made during the playbook are captured in ADRs or governed registries.

Completion Evidence

The playbook is complete when accountable owners approve the outputs, downstream teams can implement from them, testable acceptance criteria exist, and critical uncertainty is either resolved or formally controlled.

Playbook — Domain Discovery

Purpose

Turn interviews, existing systems, regulations, and process maps into a bounded domain model.

When to Use

Use this playbook when creating or materially changing a GTOS domain, platform capability, integration, data product, AI capability, security control, migration, or operational process. The facilitator SHALL adapt detail to impact without removing foundational questions.

Preparation

Identify sponsor, accountable owner, participants, decisions required, source evidence, current constraints, affected jurisdictions, and the deadline. Distribute existing domain models, contracts, incidents, policies, and metrics before the session.

Working Steps

1. Collect business language and examples.
2. Identify authoritative propositions.
3. Map state and lifecycle.
4. Identify decisions and policies.
5. Separate current workaround from required business law.

Facilitation Questions

- What exists in business reality independently of our software?
- Who owns each proposition and under what authority?
- Which identifiers remain stable through change?
- Which time dimensions matter?
- What state and invariants must be protected?
- Which messages are Commands, Events, Observations, Recommendations, or Decisions?
- What evidence is available and what remains uncertain?
- What Intent authorizes execution?
- How are correction, replay, recovery, and dispute handled?

Outputs

- domain charter
- ubiquitous language
- aggregate candidates
- invariant list
- open semantic questions

Quality Controls

Outputs SHALL use GTOS terminology, identify Canon dependencies, distinguish authority from technical ownership, and include unresolved questions. Decisions made during the playbook are captured in ADRs or governed registries.

Completion Evidence

The playbook is complete when accountable owners approve the outputs, downstream teams can implement from them, testable acceptance criteria exist, and critical uncertainty is either resolved or formally controlled.

Playbook — Event Storming and State Modeling

Purpose

Discover business occurrences while avoiding the event-as-command anti-pattern.

When to Use

Use this playbook when creating or materially changing a GTOS domain, platform capability, integration, data product, AI capability, security control, migration, or operational process. The facilitator SHALL adapt detail to impact without removing foundational questions.

Preparation

Identify sponsor, accountable owner, participants, decisions required, source evidence, current constraints, affected jurisdictions, and the deadline. Distribute existing domain models, contracts, incidents, policies, and metrics before the session.

Working Steps

1. Start from outcomes.
2. Identify states and invariants.
3. Separate commands, events, decisions, and timers.
4. Add actors and authority.
5. Test late and contradictory information.

Facilitation Questions

- What exists in business reality independently of our software?
- Who owns each proposition and under what authority?
- Which identifiers remain stable through change?
- Which time dimensions matter?
- What state and invariants must be protected?
- Which messages are Commands, Events, Observations, Recommendations, or Decisions?
- What evidence is available and what remains uncertain?
- What Intent authorizes execution?
- How are correction, replay, recovery, and dispute handled?

Outputs

- state diagrams
- command catalogue
- event catalogue
- policy hotspots
- workflow candidates

Quality Controls

Outputs SHALL use GTOS terminology, identify Canon dependencies, distinguish authority from technical ownership, and include unresolved questions. Decisions made during the playbook are captured in ADRs or governed registries.

Completion Evidence

The playbook is complete when accountable owners approve the outputs, downstream teams can implement from them, testable acceptance criteria exist, and critical uncertainty is either resolved or formally controlled.

Playbook — Incident Learning

Purpose

Convert incidents into architecture and operational improvement without rewriting history.

When to Use

Use this playbook when creating or materially changing a GTOS domain, platform capability, integration, data product, AI capability, security control, migration, or operational process. The facilitator SHALL adapt detail to impact without removing foundational questions.

Preparation

Identify sponsor, accountable owner, participants, decisions required, source evidence, current constraints, affected jurisdictions, and the deadline. Distribute existing domain models, contracts, incidents, policies, and metrics before the session.

Working Steps

1. Preserve evidence.
2. Reconstruct timeline and knowledge.
3. Separate trigger from conditions.
4. Analyze decisions and controls.
5. Define actions.
6. Verify completion.

Facilitation Questions

- What exists in business reality independently of our software?
- Who owns each proposition and under what authority?
- Which identifiers remain stable through change?
- Which time dimensions matter?
- What state and invariants must be protected?
- Which messages are Commands, Events, Observations, Recommendations, or Decisions?
- What evidence is available and what remains uncertain?
- What Intent authorizes execution?
- How are correction, replay, recovery, and dispute handled?

Outputs

- incident record
- causal analysis
- decision review
- remediation backlog
- control verification

Quality Controls

Outputs SHALL use GTOS terminology, identify Canon dependencies, distinguish authority from technical ownership, and include unresolved questions. Decisions made during the playbook are captured in ADRs or governed registries.

Completion Evidence

The playbook is complete when accountable owners approve the outputs, downstream teams can implement from them, testable acceptance criteria exist, and critical uncertainty is either resolved or formally controlled.

Playbook — Migration and Cutover

Purpose

Move meaning, history, consumers, and operations safely to a new implementation.

When to Use

Use this playbook when creating or materially changing a GTOS domain, platform capability, integration, data product, AI capability, security control, migration, or operational process. The facilitator SHALL adapt detail to impact without removing foundational questions.

Preparation

Identify sponsor, accountable owner, participants, decisions required, source evidence, current constraints, affected jurisdictions, and the deadline. Distribute existing domain models, contracts, incidents, policies, and metrics before the session.

Working Steps

1. Map source and target semantics.
2. Preserve identity and time.
3. Run dual-read or controlled dual-write.
4. Reconcile propositions.
5. Stage consumers.
6. Cut over and retire.

Facilitation Questions

- What exists in business reality independently of our software?
- Who owns each proposition and under what authority?
- Which identifiers remain stable through change?
- Which time dimensions matter?
- What state and invariants must be protected?
- Which messages are Commands, Events, Observations, Recommendations, or Decisions?
- What evidence is available and what remains uncertain?
- What Intent authorizes execution?
- How are correction, replay, recovery, and dispute handled?

Outputs

- migration mapping
- reconciliation report
- consumer readiness
- rollback plan
- retirement evidence

Quality Controls

Outputs SHALL use GTOS terminology, identify Canon dependencies, distinguish authority from technical ownership, and include unresolved questions. Decisions made during the playbook are captured in ADRs or governed registries.

Completion Evidence

The playbook is complete when accountable owners approve the outputs, downstream teams can implement from them, testable acceptance criteria exist, and critical uncertainty is either resolved or formally controlled.

Playbook — Partner Onboarding

Purpose

Onboard an external organization through identity, Relationship, contracts, security, certification, and support.

When to Use

Use this playbook when creating or materially changing a GTOS domain, platform capability, integration, data product, AI capability, security control, migration, or operational process. The facilitator SHALL adapt detail to impact without removing foundational questions.

Preparation

Identify sponsor, accountable owner, participants, decisions required, source evidence, current constraints, affected jurisdictions, and the deadline. Distribute existing domain models, contracts, incidents, policies, and metrics before the session.

Working Steps

1. Verify organizational identity.
2. Establish legal and technical relationship.
3. Exchange credentials.
4. Certify contracts.
5. Test failure and reconciliation.
6. Activate monitoring.

Facilitation Questions

- What exists in business reality independently of our software?
- Who owns each proposition and under what authority?
- Which identifiers remain stable through change?
- Which time dimensions matter?
- What state and invariants must be protected?
- Which messages are Commands, Events, Observations, Recommendations, or Decisions?
- What evidence is available and what remains uncertain?
- What Intent authorizes execution?
- How are correction, replay, recovery, and dispute handled?

Outputs

- partner record
- mandates and scopes
- contract profile
- test evidence
- support contacts
- offboarding plan

Quality Controls

Outputs SHALL use GTOS terminology, identify Canon dependencies, distinguish authority from technical ownership, and include unresolved questions. Decisions made during the playbook are captured in ADRs or governed registries.

Completion Evidence

The playbook is complete when accountable owners approve the outputs, downstream teams can implement from them, testable acceptance criteria exist, and critical uncertainty is either resolved or formally controlled.

Playbook — Release Readiness

Purpose

Determine whether a release can safely carry real trade effects.

When to Use

Use this playbook when creating or materially changing a GTOS domain, platform capability, integration, data product, AI capability, security control, migration, or operational process. The facilitator SHALL adapt detail to impact without removing foundational questions.

Preparation

Identify sponsor, accountable owner, participants, decisions required, source evidence, current constraints, affected jurisdictions, and the deadline. Distribute existing domain models, contracts, incidents, policies, and metrics before the session.

Working Steps

1. Verify contracts and migrations.
2. Confirm security and policy.
3. Review slo and capacity.
4. Validate support and incident roles.
5. Test rollback and reconciliation.
6. Communicate partner changes.

Facilitation Questions

- What exists in business reality independently of our software?
- Who owns each proposition and under what authority?
- Which identifiers remain stable through change?
- Which time dimensions matter?
- What state and invariants must be protected?
- Which messages are Commands, Events, Observations, Recommendations, or Decisions?
- What evidence is available and what remains uncertain?
- What Intent authorizes execution?
- How are correction, replay, recovery, and dispute handled?

Outputs

- readiness checklist
- go/no-go record
- known issues
- rollback criteria
- support schedule

Quality Controls

Outputs SHALL use GTOS terminology, identify Canon dependencies, distinguish authority from technical ownership, and include unresolved questions. Decisions made during the playbook are captured in ADRs or governed registries.

Completion Evidence

The playbook is complete when accountable owners approve the outputs, downstream teams can implement from them, testable acceptance criteria exist, and critical uncertainty is either resolved or formally controlled.

Playbook — Security Threat Modeling

Purpose

Analyze threats across identities, partners, domains, AI, data, and operational trust boundaries.

When to Use

Use this playbook when creating or materially changing a GTOS domain, platform capability, integration, data product, AI capability, security control, migration, or operational process. The facilitator SHALL adapt detail to impact without removing foundational questions.

Preparation

Identify sponsor, accountable owner, participants, decisions required, source evidence, current constraints, affected jurisdictions, and the deadline. Distribute existing domain models, contracts, incidents, policies, and metrics before the session.

Working Steps

1. Map assets and trust boundaries.
2. Enumerate abuse cases.
3. Analyze authority confusion.
4. Test data and tool exfiltration.
5. Design detection and containment.
6. Assign residual risk.

Facilitation Questions

- What exists in business reality independently of our software?
- Who owns each proposition and under what authority?
- Which identifiers remain stable through change?
- Which time dimensions matter?
- What state and invariants must be protected?
- Which messages are Commands, Events, Observations, Recommendations, or Decisions?
- What evidence is available and what remains uncertain?
- What Intent authorizes execution?
- How are correction, replay, recovery, and dispute handled?

Outputs

- threat model
- control map
- security tests
- risk acceptance
- incident playbook

Quality Controls

Outputs SHALL use GTOS terminology, identify Canon dependencies, distinguish authority from technical ownership, and include unresolved questions. Decisions made during the playbook are captured in ADRs or governed registries.

Completion Evidence

The playbook is complete when accountable owners approve the outputs, downstream teams can implement from them, testable acceptance criteria exist, and critical uncertainty is either resolved or formally controlled.

Playbook — SLO and Runbook Design

Purpose

Translate business criticality into measurable reliability and executable operations.

When to Use

Use this playbook when creating or materially changing a GTOS domain, platform capability, integration, data product, AI capability, security control, migration, or operational process. The facilitator SHALL adapt detail to impact without removing foundational questions.

Preparation

Identify sponsor, accountable owner, participants, decisions required, source evidence, current constraints, affected jurisdictions, and the deadline. Distribute existing domain models, contracts, incidents, policies, and metrics before the session.

Working Steps

1. Identify user journeys.
2. Choose service indicators.
3. Set objectives and error budgets.
4. Define alerts.
5. Write runbooks.
6. Exercise recovery.

Facilitation Questions

- What exists in business reality independently of our software?
- Who owns each proposition and under what authority?
- Which identifiers remain stable through change?
- Which time dimensions matter?
- What state and invariants must be protected?
- Which messages are Commands, Events, Observations, Recommendations, or Decisions?
- What evidence is available and what remains uncertain?
- What Intent authorizes execution?
- How are correction, replay, recovery, and dispute handled?

Outputs

- SLO catalogue
- dashboard
- alert map
- runbooks
- exercise evidence

Quality Controls

Outputs SHALL use GTOS terminology, identify Canon dependencies, distinguish authority from technical ownership, and include unresolved questions. Decisions made during the playbook are captured in ADRs or governed registries.

Completion Evidence

The playbook is complete when accountable owners approve the outputs, downstream teams can implement from them, testable acceptance criteria exist, and critical uncertainty is either resolved or formally controlled.

Playbook — Truth Ownership Workshop

Purpose

Assign proposition-level ownership across organizations and systems.

When to Use

Use this playbook when creating or materially changing a GTOS domain, platform capability, integration, data product, AI capability, security control, migration, or operational process. The facilitator SHALL adapt detail to impact without removing foundational questions.

Preparation

Identify sponsor, accountable owner, participants, decisions required, source evidence, current constraints, affected jurisdictions, and the deadline. Distribute existing domain models, contracts, incidents, policies, and metrics before the session.

Working Steps

1. List critical propositions.
2. Identify legal and operational authorities.
3. Separate source from replica.
4. Model disputes and corrections.
5. Publish ownership decisions.

Facilitation Questions

- What exists in business reality independently of our software?
- Who owns each proposition and under what authority?
- Which identifiers remain stable through change?
- Which time dimensions matter?
- What state and invariants must be protected?
- Which messages are Commands, Events, Observations, Recommendations, or Decisions?
- What evidence is available and what remains uncertain?
- What Intent authorizes execution?
- How are correction, replay, recovery, and dispute handled?

Outputs

- truth ownership matrix
- conflict-resolution path
- replica catalogue
- authority gaps

Quality Controls

Outputs SHALL use GTOS terminology, identify Canon dependencies, distinguish authority from technical ownership, and include unresolved questions. Decisions made during the playbook are captured in ADRs or governed registries.

Completion Evidence

The playbook is complete when accountable owners approve the outputs, downstream teams can implement from them, testable acceptance criteria exist, and critical uncertainty is either resolved or formally controlled.

Delivery Roles and Responsibilities

Role	Primary Accountability	SHALL Approve
Executive Sponsor	transformation outcome, funding, organizational authority	mandate, scope, major risk
Architecture Council	Canon, Constitution, cross-domain coherence	normative changes and major exceptions
Chief Architect	end-to-end reference architecture	architecture baseline and release conformance
Domain Owner	authoritative propositions and business outcomes	domain charter, invariants, decisions
Product Manager	value, roadmap, adoption, service outcomes	product scope and measurable acceptance
Domain Architect	domain model, contracts, integration	aggregates, Commands, Events, policies
Data Owner/Steward	data meaning, quality, retention, access	data contract and quality rules
AI Owner	AI purpose, evaluation, outcome, and safety	mandate, evaluation, deployment
Security Architect	threat, identity, policy, privacy, incident controls	security design and residual risk
Platform Owner	shared capability service quality	platform contracts and SLOs
Engineering Lead	implementation quality and delivery evidence	technical design, tests, release artifact
SRE/Operations Owner	reliability, support, recovery, incident readiness	SLO, runbook, recovery evidence
Partner Integration Lead	external participant readiness	certification and cutover

Accountability Rule

Approval is not attendance. An approver confirms that the artifact is correct within their authority and that unresolved risks are explicit. One person MAY hold multiple roles in a small implementation, but conflicting duties and high-impact approvals SHOULD remain independently reviewed.

Stage 0 — Mandate and Scope

Objective

Establish the transformation mandate, sponsors, business boundaries, authority, expected outcomes, and funding model.

Entry Conditions

The preceding stage has approved evidence, open risks are visible, accountable roles are assigned, and the scope is versioned. Urgent delivery MAY begin earlier only through an explicit exception with compensating controls.

Required Work

- 1. Confirm business outcome and affected value streams.
- 2. Identify Canon and constitutional dependencies.
- 3. Resolve truth ownership, identity, time, state, authority, and evidence questions.
- 4. Produce the stage artifacts and validate them with downstream teams.
- 5. Record consequential decisions and rejected alternatives.
- 6. Evaluate security, privacy, AI, resilience, migration, and operations implications.
- 7. Demonstrate the completion criteria through evidence.

Mandatory Artifacts

- executive mandate
- scope statement
- baseline metrics
- governance charter
- initial risk register

Review Table

Review Area	Question	Evidence
Business	Is the intended outcome measurable?	metric definition and baseline
Semantics	Are concepts and ownership unambiguous?	glossary, registry, domain model
Authority	Who may assert, decide, delegate, and execute?	authority and policy model
Time and History	Can historical truth and knowledge be reconstructed?	temporal model and tests
Integration	Are contracts versioned and semantically safe?	API, Event, and data contracts
Security	Are identity, purpose, jurisdiction, and threats controlled?	threat model and policies
Operations	Can the capability be observed, recovered, and reconciled?	SLOs, runbooks, recovery test

Stop Conditions

- **STOP:** No accountable sponsor.
- **STOP:** Scope defined as a technology replacement only.
- **STOP:** No measurable business baseline.

Exit Criteria

- all mandatory artifacts are approved or have explicit exceptions;
- high-impact risks have owners and controls;
- traceability to Canon is recorded;
- delivery teams understand the next-stage obligations;
- no critical semantic, authority, security, or recovery uncertainty remains hidden.

Accountable Roles

The stage owner coordinates completion. Domain owners approve meaning and truth. Security, data, AI, and operations approve their control areas. The Architecture Council resolves cross-domain conflicts.

Stage 1 — Reference Alignment

Objective

Align stakeholders on the GTOS Reference Theory, Canon, Constitution, terminology, and conformance model.

Entry Conditions

The preceding stage has approved evidence, open risks are visible, accountable roles are assigned, and the scope is versioned. Urgent delivery MAY begin earlier only through an explicit exception with compensating controls.

Required Work

1. Confirm business outcome and affected value streams.
2. Identify Canon and constitutional dependencies.
3. Resolve truth ownership, identity, time, state, authority, and evidence questions.
4. Produce the stage artifacts and validate them with downstream teams.
5. Record consequential decisions and rejected alternatives.
6. Evaluate security, privacy, AI, resilience, migration, and operations implications.
7. Demonstrate the completion criteria through evidence.

Mandatory Artifacts

- concept workshops
- approved glossary
- Canon dependency map
- constitutional exception log

Review Table

Review Area	Question	Evidence
Business	Is the intended outcome measurable?	metric definition and baseline
Semantics	Are concepts and ownership unambiguous?	glossary, registry, domain model
Authority	Who may assert, decide, delegate, and execute?	authority and policy model
Time and History	Can historical truth and knowledge be reconstructed?	temporal model and tests
Integration	Are contracts versioned and semantically safe?	API, Event, and data contracts
Security	Are identity, purpose, jurisdiction, and threats controlled?	threat model and policies
Operations	Can the capability be observed, recovered, and reconciled?	SLOs, runbooks, recovery test

Stop Conditions

- **STOP:** Domain terms conflict with Canon.
- **STOP:** Authority remains implicit.
- **STOP:** Event and Command semantics are mixed.

Exit Criteria

- all mandatory artifacts are approved or have explicit exceptions;
- high-impact risks have owners and controls;
- traceability to Canon is recorded;
- delivery teams understand the next-stage obligations;
- no critical semantic, authority, security, or recovery uncertainty remains hidden.

Accountable Roles

The stage owner coordinates completion. Domain owners approve meaning and truth. Security, data, AI, and operations approve their control areas. The Architecture Council resolves cross-domain conflicts.

Stage 2 — Business and Domain Design

Objective

Define capabilities, value streams, domain boundaries, truth owners, decisions, policies, and service outcomes.

Entry Conditions

The preceding stage has approved evidence, open risks are visible, accountable roles are assigned, and the scope is versioned. Urgent delivery MAY begin earlier only through an explicit exception with compensating controls.

Required Work

1. Confirm business outcome and affected value streams.
2. Identify Canon and constitutional dependencies.
3. Resolve truth ownership, identity, time, state, authority, and evidence questions.
4. Produce the stage artifacts and validate them with downstream teams.
5. Record consequential decisions and rejected alternatives.
6. Evaluate security, privacy, AI, resilience, migration, and operations implications.
7. Demonstrate the completion criteria through evidence.

Mandatory Artifacts

- capability map
- value-stream maps
- domain charters
- truth ownership registry
- decision-right catalogue

Review Table

Review Area	Question	Evidence
Business	Is the intended outcome measurable?	metric definition and baseline
Semantics	Are concepts and ownership unambiguous?	glossary, registry, domain model
Authority	Who may assert, decide, delegate, and execute?	authority and policy model
Time and History	Can historical truth and knowledge be reconstructed?	temporal model and tests
Integration	Are contracts versioned and semantically safe?	API, Event, and data contracts
Security	Are identity, purpose, jurisdiction, and threats controlled?	threat model and policies
Operations	Can the capability be observed, recovered, and reconciled?	SLOs, runbooks, recovery test

Stop Conditions

- **STOP:** Shared database drives boundaries.

- **STOP:** No owner for critical propositions.
- **STOP:** Capabilities lack outcome measures.

Exit Criteria

- all mandatory artifacts are approved or have explicit exceptions;
- high-impact risks have owners and controls;
- traceability to Canon is recorded;
- delivery teams understand the next-stage obligations;
- no critical semantic, authority, security, or recovery uncertainty remains hidden.

Accountable Roles

The stage owner coordinates completion. Domain owners approve meaning and truth. Security, data, AI, and operations approve their control areas. The Architecture Council resolves cross-domain conflicts.

Stage 3 — Contract and Data Design

Objective

Specify APIs, Commands, Events, schemas, temporal rules, data products, quality, lineage, retention, and migration.

Entry Conditions

The preceding stage has approved evidence, open risks are visible, accountable roles are assigned, and the scope is versioned. Urgent delivery MAY begin earlier only through an explicit exception with compensating controls.

Required Work

- 1. Confirm business outcome and affected value streams.
- 2. Identify Canon and constitutional dependencies.
- 3. Resolve truth ownership, identity, time, state, authority, and evidence questions.
- 4. Produce the stage artifacts and validate them with downstream teams.
- 5. Record consequential decisions and rejected alternatives.
- 6. Evaluate security, privacy, AI, resilience, migration, and operations implications.
- 7. Demonstrate the completion criteria through evidence.

Mandatory Artifacts

- contract catalogue
- schema registry
- event catalogue
- data contracts
- migration plan

Review Table

Review Area	Question	Evidence
Business	Is the intended outcome measurable?	metric definition and baseline
Semantics	Are concepts and ownership unambiguous?	glossary, registry, domain model
Authority	Who may assert, decide, delegate, and execute?	authority and policy model
Time and History	Can historical truth and knowledge be reconstructed?	temporal model and tests
Integration	Are contracts versioned and semantically safe?	API, Event, and data contracts
Security	Are identity, purpose, jurisdiction, and threats controlled?	threat model and policies
Operations	Can the capability be observed, recovered, and reconciled?	SLOs, runbooks, recovery test

Stop Conditions

- **STOP:** Schema compatibility confused with semantic compatibility.
- **STOP:** Historical meaning not preserved.
- **STOP:** Projections claim authority.

Exit Criteria

- all mandatory artifacts are approved or have explicit exceptions;
- high-impact risks have owners and controls;
- traceability to Canon is recorded;
- delivery teams understand the next-stage obligations;
- no critical semantic, authority, security, or recovery uncertainty remains hidden.

Accountable Roles

The stage owner coordinates completion. Domain owners approve meaning and truth. Security, data, AI, and operations approve their control areas. The Architecture Council resolves cross-domain conflicts.

Stage 4 — Security, AI, and Operational Design

Objective

Complete threat models, identity, policy, agent mandates, evidence controls, SLOs, runbooks, and recovery design.

Entry Conditions

The preceding stage has approved evidence, open risks are visible, accountable roles are assigned, and the scope is versioned. Urgent delivery MAY begin earlier only through an explicit exception with compensating controls.

Required Work

- 1. Confirm business outcome and affected value streams.
- 2. Identify Canon and constitutional dependencies.
- 3. Resolve truth ownership, identity, time, state, authority, and evidence questions.
- 4. Produce the stage artifacts and validate them with downstream teams.
- 5. Record consequential decisions and rejected alternatives.
- 6. Evaluate security, privacy, AI, resilience, migration, and operations implications.
- 7. Demonstrate the completion criteria through evidence.

Mandatory Artifacts

- threat model
- authorization model
- AI evaluation plan
- SLO catalogue
- recovery and incident plans

Review Table

Review Area	Question	Evidence
Business	Is the intended outcome measurable?	metric definition and baseline
Semantics	Are concepts and ownership unambiguous?	glossary, registry, domain model
Authority	Who may assert, decide, delegate, and execute?	authority and policy model
Time and History	Can historical truth and knowledge be reconstructed?	temporal model and tests
Integration	Are contracts versioned and semantically safe?	API, Event, and data contracts
Security	Are identity, purpose, jurisdiction, and threats controlled?	threat model and policies
Operations	Can the capability be observed, recovered, and reconciled?	SLOs, runbooks, recovery test

Stop Conditions

- **STOP:** AI tool use bypasses policy.

- **STOP:** No decision-time evidence reconstruction.
- **STOP:** Recovery lacks reconciliation.

Exit Criteria

- all mandatory artifacts are approved or have explicit exceptions;
- high-impact risks have owners and controls;
- traceability to Canon is recorded;
- delivery teams understand the next-stage obligations;
- no critical semantic, authority, security, or recovery uncertainty remains hidden.

Accountable Roles

The stage owner coordinates completion. Domain owners approve meaning and truth. Security, data, AI, and operations approve their control areas. The Architecture Council resolves cross-domain conflicts.

Stage 5 — Build and Verification

Objective

Implement through small vertical slices with invariant, contract, policy, security, resilience, and operational testing.

Entry Conditions

The preceding stage has approved evidence, open risks are visible, accountable roles are assigned, and the scope is versioned. Urgent delivery MAY begin earlier only through an explicit exception with compensating controls.

Required Work

1. Confirm business outcome and affected value streams.
2. Identify Canon and constitutional dependencies.
3. Resolve truth ownership, identity, time, state, authority, and evidence questions.
4. Produce the stage artifacts and validate them with downstream teams.
5. Record consequential decisions and rejected alternatives.
6. Evaluate security, privacy, AI, resilience, migration, and operations implications.
7. Demonstrate the completion criteria through evidence.

Mandatory Artifacts

- working vertical slices
- test evidence
- deployment artifacts
- observability dashboards
- architecture decision records

Review Table

Review Area	Question	Evidence
Business	Is the intended outcome measurable?	metric definition and baseline
Semantics	Are concepts and ownership unambiguous?	glossary, registry, domain model
Authority	Who may assert, decide, delegate, and execute?	authority and policy model
Time and History	Can historical truth and knowledge be reconstructed?	temporal model and tests
Integration	Are contracts versioned and semantically safe?	API, Event, and data contracts
Security	Are identity, purpose, jurisdiction, and threats controlled?	threat model and policies
Operations	Can the capability be observed, recovered, and reconciled?	SLOs, runbooks, recovery test

Stop Conditions

- **STOP:** Testing focuses only on code paths.
- **STOP:** Telemetry lacks business meaning.
- **STOP:** Architecture deviations are undocumented.

Exit Criteria

- all mandatory artifacts are approved or have explicit exceptions;
- high-impact risks have owners and controls;
- traceability to Canon is recorded;
- delivery teams understand the next-stage obligations;
- no critical semantic, authority, security, or recovery uncertainty remains hidden.

Accountable Roles

The stage owner coordinates completion. Domain owners approve meaning and truth. Security, data, AI, and operations approve their control areas. The Architecture Council resolves cross-domain conflicts.

Stage 6 — Controlled Launch

Objective

Release through staged exposure, partner readiness, data reconciliation, monitoring, rollback, and command authority controls.

Entry Conditions

The preceding stage has approved evidence, open risks are visible, accountable roles are assigned, and the scope is versioned. Urgent delivery MAY begin earlier only through an explicit exception with compensating controls.

Required Work

- 1. Confirm business outcome and affected value streams.
- 2. Identify Canon and constitutional dependencies.
- 3. Resolve truth ownership, identity, time, state, authority, and evidence questions.
- 4. Produce the stage artifacts and validate them with downstream teams.
- 5. Record consequential decisions and rejected alternatives.
- 6. Evaluate security, privacy, AI, resilience, migration, and operations implications.
- 7. Demonstrate the completion criteria through evidence.

Mandatory Artifacts

- launch readiness review
- partner certification
- cutover evidence
- rollback criteria
- support rota

Review Table

Review Area	Question	Evidence
Business	Is the intended outcome measurable?	metric definition and baseline
Semantics	Are concepts and ownership unambiguous?	glossary, registry, domain model
Authority	Who may assert, decide, delegate, and execute?	authority and policy model
Time and History	Can historical truth and knowledge be reconstructed?	temporal model and tests
Integration	Are contracts versioned and semantically safe?	API, Event, and data contracts
Security	Are identity, purpose, jurisdiction, and threats controlled?	threat model and policies
Operations	Can the capability be observed, recovered, and reconciled?	SLOs, runbooks, recovery test

Stop Conditions

- **STOP:** Unverified migration.
- **STOP:** No consumer compatibility inventory.
- **STOP:** Rollback would erase new business facts.

Exit Criteria

- all mandatory artifacts are approved or have explicit exceptions;
- high-impact risks have owners and controls;
- traceability to Canon is recorded;
- delivery teams understand the next-stage obligations;
- no critical semantic, authority, security, or recovery uncertainty remains hidden.

Accountable Roles

The stage owner coordinates completion. Domain owners approve meaning and truth. Security, data, AI, and operations approve their control areas. The Architecture Council resolves cross-domain conflicts.

Stage 7 — Operate and Evolve

Objective

Operate against SLOs, measure outcomes, manage incidents, review exceptions, retire old semantics, and expand the portfolio.

Entry Conditions

The preceding stage has approved evidence, open risks are visible, accountable roles are assigned, and the scope is versioned. Urgent delivery MAY begin earlier only through an explicit exception with compensating controls.

Required Work

1. Confirm business outcome and affected value streams.
2. Identify Canon and constitutional dependencies.
3. Resolve truth ownership, identity, time, state, authority, and evidence questions.
4. Produce the stage artifacts and validate them with downstream teams.
5. Record consequential decisions and rejected alternatives.
6. Evaluate security, privacy, AI, resilience, migration, and operations implications.
7. Demonstrate the completion criteria through evidence.

Mandatory Artifacts

- SLO reports
- decision and AI outcome reviews
- incident learning
- deprecation progress
- benefit realization

Review Table

Review Area	Question	Evidence
Business	Is the intended outcome measurable?	metric definition and baseline
Semantics	Are concepts and ownership unambiguous?	glossary, registry, domain model
Authority	Who may assert, decide, delegate, and execute?	authority and policy model
Time and History	Can historical truth and knowledge be reconstructed?	temporal model and tests
Integration	Are contracts versioned and semantically safe?	API, Event, and data contracts
Security	Are identity, purpose, jurisdiction, and threats controlled?	threat model and policies
Operations	Can the capability be observed, recovered, and reconciled?	SLOs, runbooks, recovery test

Stop Conditions

- **STOP:** Old and new truth owners coexist indefinitely.
- **STOP:** Repeated exceptions are normalized.
- **STOP:** Benefits are not measured.

Exit Criteria

- all mandatory artifacts are approved or have explicit exceptions;
- high-impact risks have owners and controls;
- traceability to Canon is recorded;
- delivery teams understand the next-stage obligations;
- no critical semantic, authority, security, or recovery uncertainty remains hidden.

Accountable Roles

The stage owner coordinates completion. Domain owners approve meaning and truth. Security, data, AI, and operations approve their control areas. The Architecture Council resolves cross-domain conflicts.

Work Product Catalogue

Foundational Products

- Reference Theory adoption record
- Canon dependency model
- constitutional exception register
- glossary and semantic decision log

Business Products

- capability map
- value-stream model
- actor and Relationship map
- truth ownership matrix
- decision-right catalogue
- business service and outcome catalogue

Domain Products

- domain charter
- aggregate and invariant model
- lifecycle and state diagrams
- Command and Event catalogue
- policy catalogue
- API and data contracts
- domain SLOs

Data and AI Products

- conceptual and logical data models
- data contracts and lineage
- quality rules
- AI use-case and risk classification
- Agent mandate and tool registry
- evaluation and monitoring plan
- Decision Record schema

Security and Operations Products

- threat model
- identity, delegation, and authorization model
- data protection assessment
- runbooks and SLOs
- recovery and reconciliation plan
- incident and continuity exercises

Delivery Evidence

Each work product SHALL declare version, owner, status, scope, dependencies, reviewers, and approval date. Obsolete work products remain archived when required for historical reconstruction.

AI Agent Blueprint

Identity and Mandate

Every production Agent SHALL have a stable workload identity and an explicit Mandate. The Mandate defines purpose, allowed data classes, tools, decision classes, financial or operational limits, jurisdictions, validity, and escalation.

Runtime Structure

The runtime includes context assembly, policy enforcement, model routing, planning, tool execution, structured output validation, telemetry, and evidence recording.

The model does not directly own credentials. Tool gateways enforce authorization outside the model and receive the Agent identity, Mandate, user identity where applicable, purpose, and requested action.

Planning

Plans are proposals. Before any side effect, each planned step is evaluated against policy, authority, evidence, and reversibility. Plans MAY be revised when evidence changes. Hidden or unrecorded execution paths are prohibited.

Context

Context SHALL distinguish system policy, developer instructions, user requests, trusted evidence, untrusted retrieved content, tool outputs, and prior Agent memory. Untrusted content cannot override higher-authority instructions.

Structured Outputs

High-impact outputs SHALL conform to schemas. A recommendation includes evidence, confidence, alternatives, assumptions, and escalation. A Decision requires authority. An execution request includes Intent reference and idempotency.

Failure Behavior

When evidence is insufficient, the Agent asks for evidence, escalates, or declines. It SHALL NOT fabricate identifiers, documents, approvals, or tool results. Partial tool failure remains visible.

Monitoring

Metrics include task success, evidence faithfulness, tool denial, unsafe action attempts, hallucination rate, escalation rate, cost, latency, outcome quality, and drift.

Kill and Containment

Operators can disable a model route, tool, Mandate, decision class, tenant, or Agent population. Containment preserves evidence for incident analysis.

Domain Service Blueprint

Purpose

A GTOS Domain Service is a deployable capability that protects a coherent set of business invariants and authoritative propositions. It is not defined by technology, team size, or database ownership alone. The service boundary exists because a specific authority is responsible for a specific meaning.

Required Components

A conforming Domain Service SHALL include:

- a published domain purpose and non-goals;
- authoritative proposition catalogue;
- aggregate and invariant definitions;
- command handlers;
- event publication;
- query projections;
- temporal and correction behavior;
- policy and authorization integration;
- observability;
- migration and recovery procedures;
- Canon traceability.

Command Processing

The service authenticates the caller, evaluates authority and policy, validates command schema, resolves subject identity, loads the aggregate under the required consistency boundary, checks invariants, records state change, publishes Events, and returns an outcome.

Command acceptance does not imply successful execution. A Command may be rejected because authority is absent, evidence is incomplete, state is incompatible, policy denies the action, or a dependency is unavailable.

Persistence

The persistence model may use relational, document, event, graph, or mixed storage. The model SHALL preserve identity, history, temporal meaning, provenance, and correction. Storage selection SHALL not redefine the domain.

Query Model

Query projections are perspectives optimized for use cases. They SHALL identify freshness, source, and temporal semantics. A projection failure SHALL not corrupt authoritative state.

Recovery

Recovery procedures include restoring persistence, replaying Events, rebuilding projections, validating invariants, reconciling outbox or message state, and confirming downstream consumer position. Recovery evidence SHALL prove business consistency, not merely process uptime.

Deployment

A Domain Service MAY be deployed as a modular monolith, microservice, serverless capability, or managed workflow. Deployment topology is subordinate to domain authority and operational requirements.

Federated Integration Blueprint

Objective

The blueprint supports integration among independent organizations without requiring a shared internal data model or common technology stack.

Contract Boundary

Each exchange contract declares producer, authority, subject, semantic version, schema version, temporal fields, security classification, retention expectation, and compatibility policy.

Trust Establishment

Participants establish organizational identity, technical credentials, legal or contractual Relationship, allowed purposes, data scopes, and incident contacts. Trust is not inherited merely because traffic arrives through an approved network.

Exchange Modes

GTOS supports synchronous APIs, asynchronous Events, bulk manifests, evidence references, and policy queries. The mode is chosen by business latency, authority, consistency, volume, and recovery requirements.

Delivery Semantics

Transport acknowledgement means receipt by infrastructure, not acceptance as authoritative truth. Consumers validate identity, signature or channel integrity, schema, authority, subject, time, and policy.

Reconciliation

Reconciliation compares proposition identity, authority, temporal scope, and correction lineage. Record-count equality is insufficient. Disputes are represented explicitly and routed to the responsible authority.

Partner Failure

If a partner is unavailable, GTOS applies the declared continuity policy: queue, degrade, substitute evidence, require manual review, or suspend execution. The platform SHALL not invent partner approval.

Offboarding

Offboarding revokes credentials, Relationships, delegations, subscriptions, and data access while preserving required historical evidence.

Implementation Blueprints

Blueprints translate the Reference Architecture into deployable structures without binding GTOS to a particular vendor, cloud, framework, or programming language.

Reference Deployment Blueprint

Logical Zones

A reference deployment contains edge access, identity and policy, API and Event ingress, domain runtime, workflow, AI runtime, data services, evidence services, observability, and administrative control planes.

Trust Boundaries

External participants authenticate through federated identity or partner credentials. Workload identity is used internally. AI tools cross a dedicated execution gateway. Administrative access is separated from business access.

Data Planes

Authoritative domain stores remain within domain control. Event transport distributes facts. Search, analytics, and AI indexes are derived perspectives. Evidence storage applies stronger integrity and retention.

Regional Design

Regional cells limit failure and support residency. Global services hold only data appropriate for global scope. Cross-region replication declares consistency, authority, encryption, and recovery.

Control Plane

Configuration, policy, schema, model routing, secrets, and deployment are governed through versioned control planes. Control-plane failure SHALL not produce unauthorized business action.

Availability

Critical synchronous dependencies are minimized. Workflows tolerate delayed partners. Commands use idempotency. Events support replay. Degraded modes identify which decisions or executions are suspended.

Recovery

Recovery order protects identity, policy, authoritative stores, Event transport, workflows, projections, AI services, and external exchange. Reconciliation confirms business completeness.

Concept-to-Artifact Matrix

Concept	Primary Artifact	Verification
Reality vs representation	domain charter and model	expert review and scenario test
Truth ownership	proposition registry	ownership conflict test
Identity	identifier policy and resolution model	merge, split, and duplicate test
Time	temporal model	historical and decision-time query
State	state diagram and invariants	transition and property tests
Relationship	relationship aggregate	expiry, revocation, role test
Event	event contract	schema, replay, idempotency test
History	correction and retention design	reconstruction test
Knowledge	evidence and provenance model	late and contradictory evidence test
Decision	Decision Record	authority and outcome review
Intent	Intent contract	side-effect traceability test
Execution	command handler and runbook	idempotency and recovery test

Domain Dependency Matrix

Domain	Depends On	Publishes To	Critical Coupling Rule
Party	external identity authorities	all domains	consumers reference Party identity; they do not edit it
Product	classification authorities	Order, Shipment, Compliance	product version and jurisdiction are explicit
Agreement	Party, Product	Order, Finance, Insurance	obligations preserve effective time
Order	Party, Product, Agreement	Shipment, Finance	allocation does not redefine product or agreement
Shipment	Order, Party, Product	Transport, Compliance, Risk	custody and cargo identity remain distinct
Transport	Party, Shipment	Shipment, Risk	schedule and actual movement are separate
Document	all relevant domains	Compliance, Finance, Evidence	versions and signatures are immutable
Compliance	Party, Product, Shipment, Document	Case, Shipment, Risk	legal decisions remain authority-owned
Finance	Agreement, Order, Document	Risk, Evidence	approval, execution, settlement are distinct
Decision	Evidence, Policy, Authority	all domains	Decision does not directly own domain state
Evidence	all domains	Decision, Audit	derived artifacts retain source lineage

Reference Matrices

Matrices provide fast operational navigation across architecture concerns. They complement, but do not replace, the normative chapters.

Output and Authority Matrix

Output Type	Describes	Binding?	Minimum Metadata
Retrieval	existing source content	no	source, version, access context
Extraction	structured interpretation	no	source span, method, confidence
Inference	derived claim	no	premises, method, uncertainty
Prediction	possible future	no	horizon, model, calibration
Recommendation	proposed choice	no	evidence, alternatives, confidence
Decision	governed commitment	yes within scope	authority, evidence, policy, time
Intent	desired authorized effect	authorizes execution	objective, initiator, constraints
Execution Request	technical request under Intent	may cause effect	Intent, idempotency, target
Outcome	observed result	factual claim	source, time, evidence

GTOS Verification Matrix

Layer	Purpose	Examples
Definition	verify terms and ownership	glossary review, registry validation
Invariant	protect domain truth	property tests, state transition tests
Contract	protect exchange	API, Event, schema, consumer tests
Policy	protect authority	permission, obligation, delegation tests
Temporal	protect historical meaning	bitemporal and point-in-time tests
AI Evaluation	protect evidence and behavior	factuality, calibration, tool safety
Security	protect trust boundaries	threat tests, abuse cases, penetration
Resilience	protect service and truth	dependency failure, replay, recovery
Migration	protect continuity	reconciliation and semantic equivalence
Operational	protect production response	runbook and incident exercises

Correction Propagation Pattern

Problem

A corrected fact may have influenced projections, decisions, documents, payments, risk scores, or downstream participants.

Pattern

The correction Event references the original assertion and reason. An impact service identifies dependent artifacts through lineage. Each owner decides whether to recompute, appeal, notify, compensate, or retain the prior result as historically valid.

Distinctions

- A corrected Observation may change Knowledge.
- New Knowledge may justify a new Decision.
- A new Decision may create remediation Intent.
- Remediation Execution produces new Outcomes.
- None of these erase the prior chain.

Controls

Propagation SHALL be prioritized by legal and business impact. Automated recomputation SHALL not retroactively rewrite signed documents or completed decisions without authority. Consumers SHALL record correction-handling status.

Measure

Correction closure includes detection time, affected artifact count, unresolved high-impact dependencies, notification completion, and reconciliation evidence.

Decision Recording Pattern

Purpose

A Decision Record enables explanation, appeal, audit, learning, and outcome monitoring.

Minimum Record

- decision identity and type;
- subject;
- authority identity and basis;
- decision time and knowledge cutoff;
- evidence references;
- policy and rule versions;
- recommendations considered;
- alternatives;
- uncertainty;
- decision outcome;
- explanation;
- approval or escalation;
- expected effect;
- later outcome references;
- correction or appeal status.

AI-Assisted Decision

Where AI contributes, the record additionally captures model route, model version, prompt or task version, retrieval set, tool results, calibration context, and safety checks. Private model reasoning is not required. Reproducible evidence and structured rationale are required.

Outcome Learning

Decision quality SHALL be evaluated against later outcomes without rewriting what was knowable at the time. A wrong prediction, a poor decision, and an unsuccessful execution are distinct failure classes and demand different remediation.

Event Evolution Pattern

Objective

Event schemas must evolve without changing the meaning of facts already published.

Compatibility

Additive optional fields are generally compatible. Renaming, changing units, changing identifier scope, altering temporal meaning, or changing authority semantics are breaking even when JSON remains parseable.

Rules

- Event identity and historical payload remain immutable.
- Producers publish a schema version.
- Consumers declare supported versions.
- Transformations preserve original event reference.
- A corrected business fact is a new Event, not a rewritten old Event.
- Deprecated versions have owners, consumers, sunset dates, and replay plans.

Semantic Review

Reviewers SHALL ask whether a consumer built under the old meaning would reach a different business conclusion. If yes, the change is semantically breaking.

Replay

Before replaying historical Events into new consumers, teams SHALL evaluate whether new code interprets old facts using historically appropriate semantics. Reprocessing with present-day rules may produce a new derived perspective but SHALL not masquerade as the original historical result.

Evidence Chain Pattern

Purpose

Trade decisions often depend on documents, sensor data, attestations, external registries, and human statements. Evidence must remain trustworthy even when copied or transformed.

Evidence Record

An Evidence Item records source identity, collection method, original location, content hash or integrity proof, time, classification, retention, access policy, transformations, attestations, and related Claims.

Transformations

Extraction, translation, OCR, summarization, and AI interpretation create derived evidence artifacts. They SHALL reference the source and declare method and quality. A derived summary SHALL NOT replace the original evidence.

Chain of Custody

Transfers of evidence custody or storage SHALL be recorded where legal or investigative use requires it. Access and alteration attempts are auditable.

Decision Use

Decision Records reference evidence versions actually considered. Later corrected evidence does not silently change the historical Decision Record, although it may trigger appeal or remediation.

Exception and Case Management Pattern

Context

Not every trade condition can be automated safely. Missing documents, conflicting identities, damaged cargo, policy ambiguity, suspected fraud, and partner disagreement require collaborative resolution.

Case Structure

A Case contains issues, subjects, participants, authority, severity, tasks, evidence requests, decisions, communications, deadlines, and resolution.

Rules

A Case SHALL not become a second source of domain truth. It references domain facts and publishes authorized Decisions and Outcomes back to owning domains. Case notes are Observations unless formally asserted under authority.

Closure

Closure requires resolution criteria, authority, downstream remediation, and evidence completeness. Reopening creates new history. Metrics include age, blocked time, evidence wait, decision reversal, and recurrence.

AI

AI may summarize, classify, identify missing evidence, or recommend next action. It SHALL not fabricate closure evidence or silently decide high-impact disputes.

Federated Truth Pattern

Context

No single organization owns all truth in global trade. Customs owns legal determinations; carriers own operational movement facts; banks own settlement facts; traders own commercial intent and obligations within agreement scope.

Pattern

GTOS establishes proposition-level authority. Each proposition declares owner, scope, jurisdiction, effective time, evidence, and contract. Consumers combine propositions into perspectives while retaining origin and authority.

Conflict

When authorities disagree, GTOS preserves both claims and identifies the legal or contractual mechanism for resolution. The platform SHALL NOT choose a winner based solely on message arrival time, system priority, or database location.

Projection

A control-tower view may combine shipment position, customs status, payment condition, and predicted delay. The view is operationally valuable but remains a perspective. It does not become the owner of underlying facts.

Benefits

Federation reduces reconciliation ambiguity, supports participant autonomy, and permits technology diversity without semantic anarchy.

Identity Resolution Pattern

Problem

Global trade participants are represented through legal names, registration numbers, tax identifiers, customs identifiers, partner codes, account numbers, facility identifiers, and credentials. These identifiers may be incomplete, duplicated, expired, reassigned, or valid only within a jurisdiction or relationship.

Pattern

GTOS separates the Entity from every Identifier. Each Identifier records issuer, namespace, jurisdiction, validity, verification status, evidence, and confidence. Identity resolution proposes candidate matches; an authorized identity decision establishes merge, link, split, or rejection.

Required Flow

1. Receive an Observation containing one or more identifiers.
2. Normalize without discarding the original representation.
3. Retrieve candidate Entities.
4. Evaluate deterministic and probabilistic evidence.
5. Produce a Recommendation with confidence and explanation.
6. Make an authorized identity Decision when consequences are material.
7. Preserve the original Observation, Decision, and resulting Relationship.
8. Propagate correction through governed events.

Failure Controls

A high similarity score SHALL NOT automatically merge legal parties when sanctions, payments, title, or regulatory rights may be affected. Merge operations SHALL be reversible through preserved history. Consumers SHALL distinguish canonical Entity identity from partner-specific identifiers.

Canon Traceability

CANON-001, CANON-002, CANON-003, CANON-008, CANON-009, and CANON-010.

GTOS Reference Patterns

Reference Patterns demonstrate how Canon and Constitution become repeatable architecture structures. They are technology-neutral and include explicit failure controls.

Long-Running Trade Workflow Pattern

Context

International trade workflows may span weeks, involve many organizations, depend on documents and decisions, and encounter partial failure. A single distributed transaction is neither available nor desirable.

Pattern

A workflow coordinates Intent, Commands, Events, timers, evidence requests, human tasks, and compensation. Domains remain authoritative for their facts. The workflow owns coordination state only.

Example Sequence

1. An accepted Order creates Intent to fulfil.
2. Shipment planning requests transport booking.
3. The Transport Domain records booking outcome.
4. Documentation requests missing certificates.
5. Compliance evaluates evidence and may request inspection.
6. Finance confirms payment conditions.
7. Release Decision authorizes delivery execution.
8. Outcome Events close obligations.

Rules

Every step SHALL be idempotent. Timers SHALL identify business meaning, not only technical delay. Compensation SHALL be modeled as new authorized action, not deletion of history. Human intervention SHALL be first-class. Workflow retry SHALL NOT assume the prior Command failed merely because a response was lost.

Observability

Operators need workflow age, blocked reason, missing evidence, current authority dependency, retry state, and expected next outcome. Technical traces alone are insufficient.

Relationship-Aware Access Pattern

Problem

Role-based access alone cannot express trade relationships such as broker mandate, carrier custody, buyer-supplier contract, insurer coverage, customs jurisdiction, or temporary investigation assignment.

Policy Context

Authorization evaluates:

- authenticated identity;
- requested action;
- resource and classification;
- active Relationship and role;
- delegation or mandate;
- jurisdiction;
- business purpose;
- case or workflow context;
- time;
- risk and evidence requirements.

Example

A customs broker may submit a declaration for an importer only while a valid mandate exists for the applicable jurisdiction and commodity scope. Access to unrelated importer records is not implied.

Controls

Relationship validity SHALL be checked at decision time. Revocation SHALL propagate promptly. Cached authorization SHALL not outlive delegation. Policy results SHALL record version and material inputs. Emergency access requires heightened logging, expiry, and review.

Temporal Reconstruction Pattern

Purpose

Auditors, operators, and AI reviewers frequently need to know not only the current state but what was true, what was recorded, and what was knowable at a previous time.

Model

A governed record may include:

- valid-from and valid-to;
- observed-at;
- recorded-at;
- corrected-at;
- supersedes reference;
- source and authority;
- transaction sequence.

The reconstruction service accepts a subject, business-time boundary, knowledge-time boundary, and perspective. It returns the state derivable from assertions valid under those boundaries.

Example

A customs release was legally effective at 10:00. GTOS received the release at 10:08. A carrier decision made at 10:05 could not legitimately rely on the release even though a present-day query shows the cargo as released at 10:00.

Requirements

Reconstruction SHALL use the same rules and schema versions applicable to the historical period or explicitly disclose reinterpretation. Late data and corrections SHALL not be silently back-projected into a decision-time knowledge view. Queries SHALL name their temporal semantics.

Operational Use

Temporal reconstruction supports incident analysis, legal evidence, model evaluation, customer dispute resolution, correction propagation, and recovery reconciliation.

Scenario — Customs Declaration and Release

Context

An importer, broker, carrier, customs authority, and terminal coordinate declaration and release.

Authority

The importer owns commercial and product information within its scope. The broker acts through a mandate. Customs owns legal acceptance, inspection, and release determinations. The terminal owns physical gate and cargo movement observations.

Flow

A broker submits a declaration Command under a valid Relationship and delegation. Customs records DeclarationSubmitted, evaluates policy and evidence, and may produce InspectionOrdered or GoodsReleased. The carrier's operational view consumes the customs Event but remains a perspective.

Late Information

A corrected classification arrives after release. GTOS preserves the original declaration, knowledge available to customs, policy version, and release Decision. The correction triggers impact analysis for duty, license, risk, and audit. It does not rewrite the original decision into a fictional present-day evaluation.

Security

Authorization considers importer-broker relationship, jurisdiction, commodity scope, mandate validity, and purpose. Possession of a declaration identifier does not grant access.

Operational Requirement

If messaging is unavailable, customs authority may still make a decision through its own system. Later ingestion records observation and recording time, preserving that reality preceded GTOS representation.

End-to-End Trade Scenarios

Scenarios validate that the architecture remains coherent across domains, organizations, time, uncertainty, decision, and execution.

Scenario — Multi-Party Shipment Consolidation

Context

Multiple orders from different sellers are consolidated into cargo units, transported across legs, deconsolidated, and delivered to multiple buyers.

Modeling

Orders, shipments, consignments, cargo units, transport legs, and custody Relationships retain independent identity. A container is not the shipment, and a shipment is not the transport service.

Events

CargoUnitPacked, ShipmentConsolidated, CustodyTransferred, TransportDeparted, TransportArrived, and ShipmentDelivered describe distinct facts. Schedule updates are not actual movement events.

State

A consolidated shipment may contain one held consignment and several releasable consignments. State is scoped; one universal status would either block too much or release too much.

Recovery

If a message is duplicated, domain invariants and event identity prevent duplicate custody transfer. If events arrive out of order, occurrence time and sequence scope permit reconstruction while preserving receipt order.

Analytics

A control tower projects end-to-end progress but does not own order, customs, payment, or physical movement truth.

Scenario — Pharmaceutical Cold Chain

Context

A temperature-controlled pharmaceutical shipment moves from manufacturer to airport, airline, destination terminal, customs, and hospital distributor. Product stability, custody, temperature, documentation, release, and insurance are governed by different authorities.

Flow

The Product Domain owns the regulated product identity and storage specification. The Agreement Domain owns service obligations. The Shipment Domain owns cargo grouping and custody milestones. Sensors publish Observations. The platform records occurrence, observation, and recording time separately.

At 14:02 the refrigeration unit fails. A sensor observes the condition at 14:05. Connectivity transmits it at 14:09. Risk produces a recommendation at 14:10. An authorized logistics Decision at 14:12 directs transfer to qualified cold storage. Intent is issued to the terminal operator, which executes and records the Outcome.

Architectural Analysis

The sensor Observation is not automatically authoritative product damage. Product stability may require duration, temperature curve, and qualified assessment. The AI recommendation is not the Decision. The Decision Record includes the knowledge cutoff and evidence. Insurance and compliance consume the preserved chain.

Failure Handling

If the sensor is later found miscalibrated, the original Observation remains. A correction and calibration evidence are added. Decisions are reviewed using what was knowable at the time. Remediation may be unnecessary, but the earlier emergency transfer remains a real Outcome.

Canon Coverage

This scenario exercises all eleven Canons, particularly time, knowledge, decision, intent, and historical correction.

Scenario — Sanctions and Restricted-Party Screening

Context

Trade participants and beneficial owners are screened against external lists that change over time and contain uncertain identity information.

Knowledge Discipline

A screening match is a Claim or risk signal, not proof that the party is sanctioned. The system preserves list edition, matching method, identifiers, confidence, and analyst evidence.

Decision

Policy defines thresholds for automatic clear, evidence request, human review, or prohibition. A binding decision requires authority and records the knowledge cutoff.

Temporal Behavior

A party may be clear when an order is accepted and later listed before shipment. GTOS distinguishes decision time, policy effective time, and execution time. Controls may require re-screening at defined lifecycle points.

Correction

False-positive resolution records evidence and authority. It does not remove the fact that a match occurred and influenced earlier operations.

AI

AI may assist entity resolution and evidence summarization, but SHALL not promote probabilistic similarity into legal identity without the approved decision path.

Scenario — Trade Finance and Settlement

Context

A buyer, seller, bank, carrier, and insurer coordinate documentary obligations and payment.

Separation of Concepts

An invoice is evidence of a claimed obligation, not proof of settlement. A bank recommendation is not payment approval. Payment approval is a Decision. Payment Intent authorizes execution. Settlement is an Event owned by the settlement authority.

Workflow

Agreement terms define required documents and deadlines. Documents are presented with versions and signatures. Validation produces observations and findings. The bank decision uses an Evidence Set and policy. Approved payment produces Intent with amount, currency, beneficiary, limits, and idempotency key. The payment service executes and records settlement or failure.

Duplicate Protection

A lost response does not imply failed settlement. Retrying with the same idempotency key queries or resumes execution. A second business payment requires new Intent and authority.

Correction

If a document was fraudulent, investigation does not delete settlement history. New evidence may create fraud Decision, recovery Intent, insurance claim, or legal hold.

GTOS API Contract Standard

Resource and Action Semantics

APIs SHALL use domain language. A resource endpoint represents a governed concept or perspective. An action endpoint represents a Command or Decision request. API naming SHALL not imply authority the provider does not own.

Required Metadata

Requests and responses include correlation identity, caller identity through security context, version, content type, and temporal fields where relevant. Commands additionally include idempotency key, authority context, expected subject version, and Intent reference when execution is involved.

Error Model

Errors are categorized as authentication failure, authorization denial, invalid evidence, validation failure, invariant conflict, optimistic concurrency conflict, policy requirement, dependency failure, rate limit, or internal fault. Error responses SHALL not expose sensitive implementation details.

Compatibility

Adding an optional field may be compatible. Changing meaning, units, identifier scope, default policy, time semantics, or authority is breaking. Deprecation identifies consumers and sunset date.

Query Semantics

Queries SHALL declare whether they return authoritative current state, a projection, historical state, decision-time knowledge, or analytical output. Pagination ordering SHALL be stable. Filters SHALL have defined timezone and comparison semantics.

Idempotency

Commands capable of side effects SHALL support idempotency appropriate to business identity. An idempotency key maps to one logical attempt and outcome. Reuse with materially different input is rejected.

Security

Object-level and relationship-level authorization is mandatory. Bulk endpoints require scope and volume controls. Sensitive fields are minimized and purpose-controlled.

Observability

Metrics include request outcome, policy denial, invariant conflict, dependency latency, consumer version, and business effect. Logs preserve correlation without leaking secrets or prohibited personal data.

Conformance Assessment Standard

Assessment Scope

An assessment declares organization, product, domains, environments, version, jurisdictions, and exclusions.

Evidence Method

Assessors evaluate documents, registries, schemas, code, tests, configurations, operational records, incidents, and interviews. Statements without evidence do not establish conformance.

Finding Classes

- conforming;
- observation;
- minor non-conformance;
- major non-conformance;
- critical non-conformance;
- approved exception.

Critical Findings

Critical findings include unowned authoritative truth, unauthorized automated execution, erased governed history, material security bypass, inability to reconstruct high-impact decisions, or false claims of regulatory authority.

Remediation

Each finding has owner, impact, root condition, remediation, compensating control, deadline, and verification. Closure requires evidence.

Certification Boundary

Conformance applies only to the assessed scope and version. A compliant platform does not make every deployed extension compliant.

Continuous Conformance

Automated checks monitor schemas, registry completeness, policy tests, dependency status, traceability, and operational controls. Periodic review covers semantic and governance concerns not fully automatable.

GTOS Event Contract Standard

Business Naming

Event names describe completed business facts in past tense. `ShipmentReleased` is valid; `ReleaseShipment` is a Command. Technical lifecycle Events are allowed but SHALL not masquerade as business facts.

Envelope

The Event envelope includes event identity, type, subject identity, source identity and domain, occurrence time, observation time where applicable, recording time, schema version, authority reference where material, correlation, causation, integrity, and payload.

Immutability

Published Events are immutable. A correction or retraction is a new Event referencing the prior Event. Infrastructure metadata may be appended without changing business payload meaning.

Ordering

Ordering is guaranteed only within an explicitly declared scope such as aggregate identity or partition key. Consumers SHALL not assume global order. Occurrence order, publication order, and receipt order are distinct.

Delivery and Idempotency

At-least-once delivery is expected. Consumers deduplicate by Event identity and protect business invariants. A duplicate Event SHALL not create duplicate business effect.

Schema Evolution

Schemas have semantic owners. Breaking changes use a major version or a new Event type. Translators preserve original identity and provenance. Historical Events remain readable for the required retention period.

Replay

Replay is controlled, observable, and authorized. Consumers distinguish live and replay processing when operational behavior differs. Notifications and irreversible side effects SHALL not be repeated without explicit replay policy.

Privacy

Events contain the minimum necessary information. Highly sensitive payloads SHOULD use references with access control rather than broad replication.

Financial Control Standard

Separation of Duties

Creation of an obligation, approval of payment, execution of settlement, and reconciliation SHOULD be separable. Where automation combines steps, independent policy and audit controls are required.

Monetary Semantics

Amounts include currency, precision, sign meaning, tax or fee treatment, exchange-rate source, and effective time. Floating-point representation SHALL not be used for authoritative monetary calculation.

Payment Intent

Payment execution requires Intent containing beneficiary, account or instrument reference, amount, currency, purpose, authority, limits, validity, and idempotency.

Fraud and Risk

Risk signals inform policy and Decisions but do not silently change obligations. High-risk or unusual transactions may require additional evidence or dual approval.

Settlement

A provider acceptance is not settlement. Settlement state is owned by the recognized settlement authority. Reconciliation compares authoritative settlement facts to obligations and execution records.

Correction

Financial corrections use adjustment, reversal, refund, chargeback, or compensating entry according to authority and accounting meaning. Historical entries are not simply edited.

Audit

Records preserve decision authority, evidence, approval, execution, provider response, settlement, reconciliation, and exceptions.

Cross-Cutting Standards

These standards define testable requirements for APIs, Events, policy, privacy, jurisdiction, finance, observability, migration, and conformance.

Jurisdiction Model

Dimensions

Jurisdiction may derive from party establishment, facility location, goods origin or destination, transport route, data subject location, regulator authority, contract choice of law, payment system, or data-processing region.

No single country field is sufficient.

Jurisdiction Context

A jurisdiction context records applicable dimensions, evidence, effective time, uncertainty, and decision authority. It may be computed, but the computation remains traceable.

Policy Application

Policies declare jurisdiction scope and precedence. When multiple regimes apply, GTOS preserves each obligation and identifies conflict-resolution authority.

Change Over Time

Jurisdiction can change during a value stream. A shipment route change, ownership transfer, or new regulatory designation may trigger re-evaluation.

Data Architecture

Data contracts identify residency, transfer, localization, and retention constraints. Derived datasets inherit applicable restrictions unless an approved transformation changes classification.

Operations

Incident response and support access respect jurisdiction. Emergency access does not erase notification, evidence, or review obligations.

AI

Agent mandates and model routing are jurisdiction-aware. An Agent approved for one region SHALL not silently process restricted data in another.

Semantic Migration Standard

Scope

Migration includes database change, service extraction, platform replacement, partner transition, schema evolution, identity merge, policy change, and model upgrade.

Required Plan

A migration plan defines source and target semantics, ownership, mapping, temporal treatment, correction behavior, compatibility, validation, cutover, rollback, reconciliation, and retirement.

Dual Operation

During dual run, one truth owner remains explicit. Dual writes require deterministic conflict handling and reconciliation. Comparing record counts is insufficient; propositions and outcomes are compared.

Historical Data

Historical data retains original identifiers, source, time, and meaning. Transformations record lineage. Present-day rules SHALL not silently reinterpret historical facts.

Cutover

Cutover criteria include data completeness, invariant validity, consumer readiness, operational telemetry, rollback window, and unresolved exceptions.

AI and Analytics

Feature and model migrations require point-in-time validation and decision comparison. Vector embeddings or summaries SHALL not become the only retained representation of evidence.

Completion

Migration completes only after old write paths are disabled, consumers are migrated, evidence is archived, reconciliation passes, and ownership is updated.

End-to-End Observability Standard

Four Views

GTOS observability includes technical health, business flow, decision quality, and evidence integrity.

Technical Health

Metrics cover latency, errors, saturation, dependency behavior, queue age, storage, and deployment.

Business Flow

Metrics cover value-stream cycle time, state age, exception rate, blocked reason, missing evidence, command rejection, Event freshness, and completion outcome.

Decision Quality

Metrics cover recommendation acceptance, reversal, appeal, calibration, outcome quality, policy denial, human escalation, and automation suspension.

Evidence Integrity

Metrics cover missing provenance, stale source, failed signature, lineage gap, correction backlog, and reconstruction success.

Correlation

Correlation links related work but does not merge identities or authority. Trace context SHALL be sanitized at trust boundaries.

Alerting

Alerts map to an owned response and user or business impact. High-cardinality business identifiers are controlled to avoid leakage and cost.

Retention

Telemetry retention follows operational, security, privacy, and legal requirements. Audit evidence may require stronger integrity than ordinary logs.

GTOS Policy Standard

Purpose

Policy expresses permission, prohibition, obligation, required approval, threshold, routing, or evidence requirement.

Policy Anatomy

A policy has identity, owner, authority basis, scope, effective time, version, inputs, rule logic, result types, explanation template, tests, and retirement state.

Input Context

Inputs may include actor identity, organizational Relationship, delegation, resource, action, purpose, jurisdiction, time, classification, risk, evidence, and operational state.

Results

A result includes permit, deny, require approval, require evidence, or permit with obligations. The result records policy version, material inputs, reason codes, and decision time.

Separation

Policy evaluation is not always a business Decision. A policy may provide a required control used by an authorized Decision process. Where policy automatically produces a binding determination, the evaluation is recorded as a Decision with authority.

Testing

Policies require positive, negative, boundary, temporal, jurisdiction, delegation, and conflict tests. Production policy changes use staged rollout and decision comparison.

Conflict Resolution

Conflicts are resolved by explicit priority and authority, not rule order accident. More restrictive rules do not automatically win when jurisdictions or legal authorities differ; the applicable authority model determines precedence.

Explainability

Explanations identify the governing rule and material facts without exposing secrets or enabling control evasion.

Lifecycle

Policies are reviewed when law, contract, risk, business model, or evidence changes. Expired policy SHALL not remain silently active.

Privacy and Purpose Standard

Purpose Binding

Sensitive data processing SHALL identify a legitimate purpose. Access granted for one purpose does not automatically permit another.

Data Minimization

Contracts, Events, logs, AI contexts, and analytical products contain only data necessary for the declared purpose. Broad replication is discouraged when a reference or attestation is sufficient.

Data Subject and Organization Rights

Where applicable, the architecture supports access, correction, restriction, deletion, and objection while reconciling legal hold, trade evidence, fraud prevention, and contractual obligations.

Jurisdiction

Residency and transfer controls apply to primary storage, backups, logs, search indexes, vector stores, model prompts, support tooling, and incident evidence.

AI

Personal or confidential data SHALL not enter model context unless the model route, processor, region, retention, and purpose are approved. Prompt and response logging follow classification.

Deletion

Deletion is a governed workflow. It identifies authoritative sources, derived copies, caches, indexes, exports, model datasets, and legal exceptions. Historical integrity may require redaction, tokenization, restriction, or retained evidence rather than physical erasure.

Observability

Privacy controls emit evidence of policy decisions, transfer, access, deletion completion, and unresolved exceptions.

GTOS Visual Architecture Atlas

The Visual Atlas provides a high-level map of the theory, domains, delivery lifecycle, AI runtime, data lineage, security boundaries, deployment, decision chain, and correction flow. Each diagram is explanatory; the normative text and machine-readable registries remain authoritative.

How to Use the Atlas

Use the diagrams during executive orientation, architecture workshops, domain onboarding, design reviews, engineering onboarding, partner certification, and incident exercises. Every diagram should be read together with its corresponding chapter and scenario.